

GUIÓN DE PRÁCTICAS

Índice general

1. Modelado de sistemas electromecánicos	3
1.1. Resumen repaso de <i>Fundamentos de Automática</i>	3
1.1.1. Definición de funciones de transferencia	3
1.1.2. Interconexión y operaciones con bloques	3
1.1.3. Visualización de la respuesta de un sistema	4
1.1.4. Modelado de sistemas con Simulink	4
1.2. Ejercicios a realizar	6
2. PID en tiempo continuo	7
2.1. PID con Matlab	7
2.2. Ejercicio: Control PID de la fuerza aplicada sobre una masa	7
2.2.1. Cuestiones a resolver	8
2.3. Sintonizado de PID automático (Autotune)	9
2.3.1. Cuestiones	9
2.4. PID con Simulink	10
2.4.1. Cuestiones	11
3. Compensación Proporcional-Integral-Derivativa	12
3.1. Ejercicio: Sistema con compensación PID	12
3.1.1. Cuestiones	13
4. PID en tiempo discreto	14
4.1. Discretización de un controlador PID	14
4.1.1. Implementación de controlador PID en tiempo discreto	15
4.1.2. PID como un filtro IIR	15
4.1.3. Filtro pasa baja en el término derivativo	15
4.2. Tareas a realizar	16
5. Modelado de sistemas en el espacio de estados	17
5.1. Creación del modelo de un sistema continuo en el espacio de estados	17
5.1.1. Conversión de modelos entre representación interna y externa	18
5.1.2. Cálculo y visualización de la respuesta del sistema	18
5.1.3. Cuestiones	19
5.2. Controlabilidad y observabilidad de un sistema	19
5.2.1. Cuestiones	19
5.3. Modelo de estados del robot Turtlebot (opcional)	20
5.4. Cuestiones	21

6. Proyecto péndulo invertido	22
6.1. Objetivos	22
6.2. Especificaciones	22
6.3. Entregables	22
6.4. Aclaraciones	22
6.5. Ejemplo de informe	23
A. Ejemplo de informe del proyecto péndulo invertido	24
B. Manual de robot Kobuki/Turtlebot2	39

Práctica 1. Modelado de sistemas electromecánicos

1.1. Resumen repaso de *Fundamentos de Automática*

A continuación se repasan varios conceptos básicos sobre el uso de Matlab y Simulink ya vistos en la asignatura de *Fundamentos de Automática*

1.1.1. Definición de funciones de transferencia

En Matlab las funciones de transferencia se pueden definir de varias formas. Por ejemplo para la función de transferencia

$$G(s) = \frac{Y(s)}{X(s)} = \frac{5s - 3}{3s^2 + 5s - 7}$$

1. Mediante cociente de polinomios:

```
% Comentario
num = [5 -3];
den = [3 5 -7];
```

2. En función de sus ceros (z), polos (p) y ganancia (g):

```
z = [5 -2];
p = [2+i 2-i -5];
k = 10;
G = zpkm(z,p,k);
```

3. Mediante el uso de variables simbólicas:

```
s = tf('s') % s es un objeto transfer function.
G = 5*s/(3*s^2 + 7);
```

Notas de interés:

- En <https://es.mathworks.com/help/control/ref/tf.html> está disponible la documentación de la función *tf*.
- El comando *tfdata* permite obtener el numerador y el denominador de una ft.
- El comando *zpkdata* permite obtener los ceros, polos y ganancia de una ft.
- Otras funciones para la extracción de información de un ft se pueden consultar en <https://es.mathworks.com/help/ident/data-extraction.html>
- El comando *residue* permite calcular la descomposición en fracciones simples de una ft.

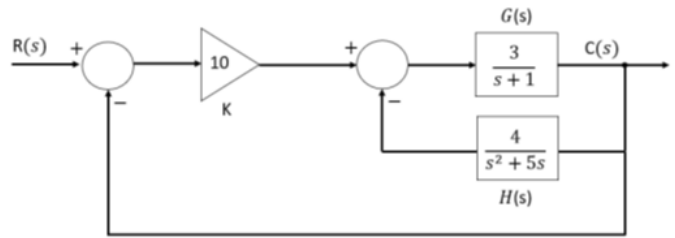
1.1.2. Interconexión y operaciones con bloques

Las funciones *feedback*, *series* y *parallel* permiten realizar conexiones entre bloques para su reducción:

```

K = tf(10, 1);
G = tf(3, [1 1]);
H = tf(4, [1 5 0]);
fb_1 = feedback(G,H,-1);
olp_1 = series(K, fb_1);
C = feedback(olp_1, 1, -1);

```



Más información sobre este tipo de funciones en: https://es.mathworks.com/help/control/model-interconnection.html?s_tid=CRUX_lftnav

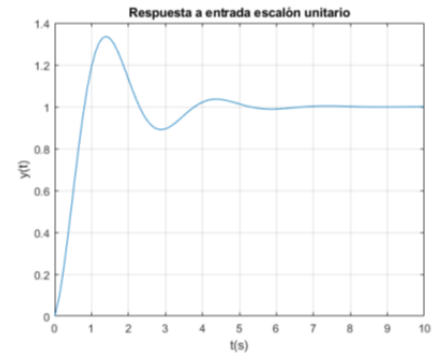
1.1.3. Visualización de la respuesta de un sistema

La función `step` permite analizar la respuesta de un sistema ante un escalón unitario. La función `plot` permite obtener la representación gráfica de la respuesta

```

G = tf([1 10], [2 3 10]);
t_sim = 10; # Tiempo de simulacion.
[y t] = step(G, t_sim);
plot(x, y);
grid;
title('Respuesta a entrada escalon de G');
xlabel('t(s)');
ylabel('y(t)');

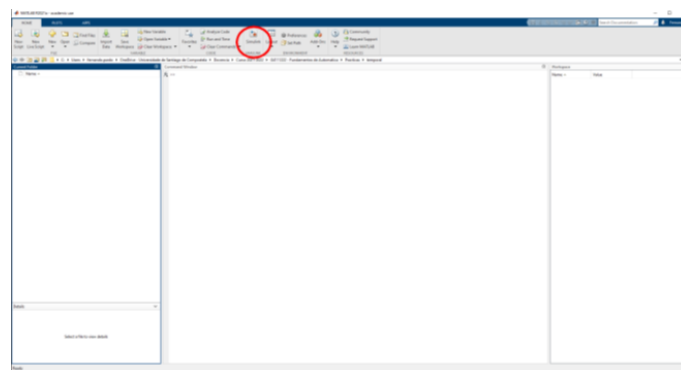
```



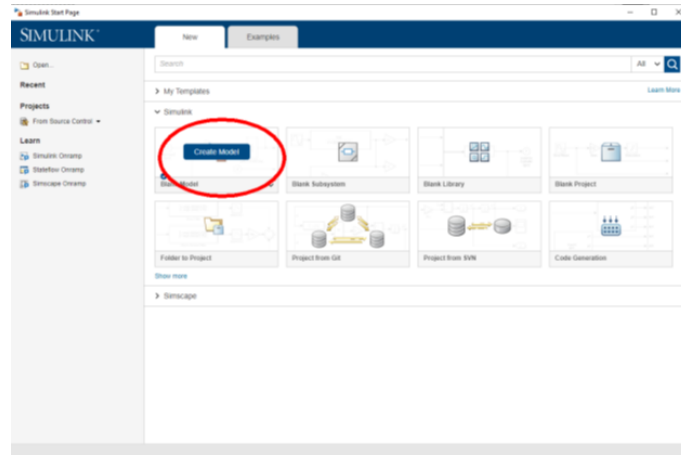
Más información relativa a herramientas análisis de repuesta y visualización en <https://es.mathworks.com/help/ident/response-visualization.html>

1.1.4. Modelado de sistemas con Simulink

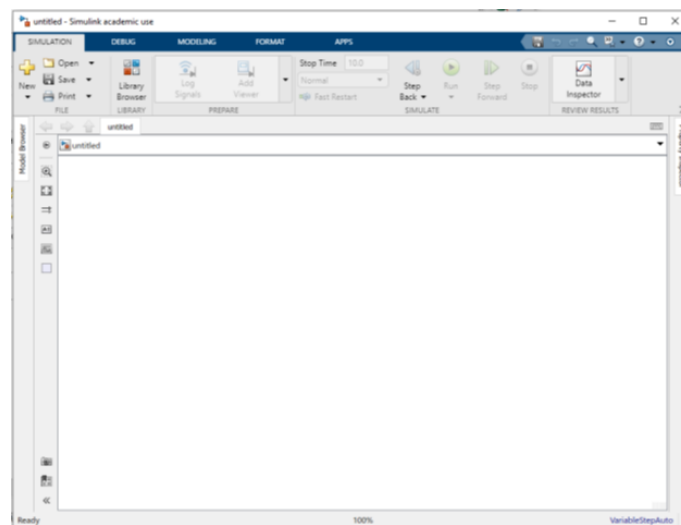
Mediante Simulink podemos crear diagramas de bloques de forma gráfica. Para ejecutarlo:



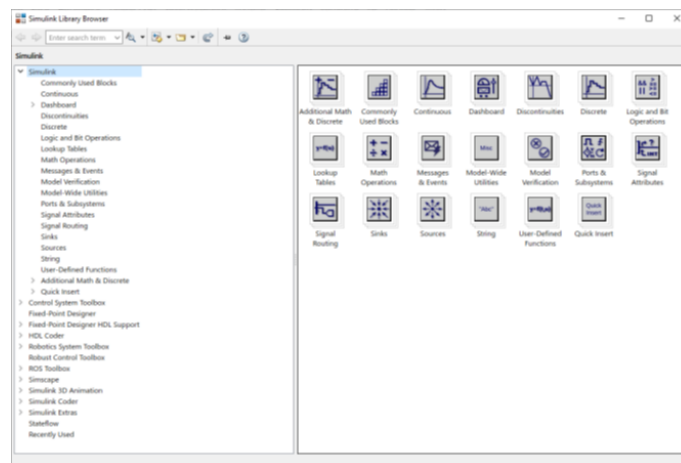
Y seleccionar “Create Blank Model”:



Se abrirá el área de trabajo donde construir el modelo de un sistema:



Para acceder a la librería de componentes pulsar “Library Browser”:



Usaremos las categorías:

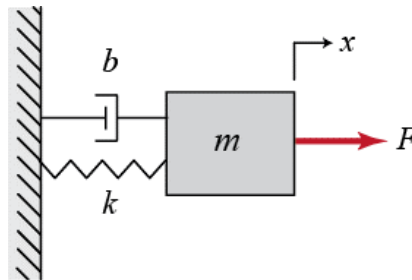
- **Continuous:** Análisis de sistemas continuos
- **Discrete:** Análisis de sistemas discretos
- **Sources:** Bloques de generación de señales

- **Sinks:** Herramientas de visualización
- **Math operation:** Operaciones matemáticas básicas

1.2. Ejercicios a realizar

1. Para el sistema de la figura obtener la función de transferencia del sistema $X(s)/F(s)$ y, usando Matlab:
 - a) Simular la respuesta del sistema ante una entrada escalón.
 - b) Obtener la respuesta del sistema ante una entrada impulso.
 - c) Obtener los ceros, polos y ganancia del sistema.
 - d) Analizar el comportamiento del sistema cuando cambian los valores de las constantes del sistema (m , k y b)

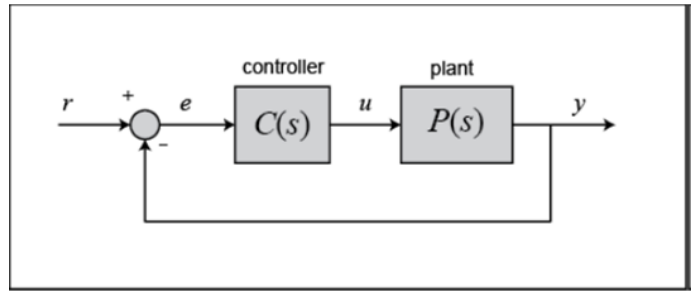
Datos iniciales: $m = 2 \text{ kg}$, $b = 10 \text{ N s m}^{-1}$, $k = 20 \text{ N m}^{-1}$



2. Para el sistema del problema 1.13 (masa en rotación con acoplamiento elástico), usando Matlab:
 - a) Simular la respuesta del sistema ante una entrada escalón.
 - b) Obtener la respuesta del sistema ante una entrada impulso.
 - c) Obtener los ceros, polos y ganancia del sistema.
 - d) Analizar el comportamiento del sistema cuando cambian los valores de las constantes del sistema (m , k y b).
3. Construye en **Simulink** el diagrama de bloques correspondiente al problema 1.9 (suspensión de un coche). Usando los valores disponibles en los apuntes de teoría para este sistema, obtener la respuesta del sistema ante una entrada escalón y una entrada impulso.
4. Construye en **Simulink** el diagrama de bloques correspondiente al sistema del problema 1.16 (motor con reductora) y obtener la respuesta de sistema ante una entrada escalón. Seleccionar valores para las constantes del sistema y observar el efecto en la respuesta de su cambio.
5. Construye con **Simulink** el diagrama de bloques del problema 1.19 y responder a las preguntas del problema.

Práctica 2. PID en tiempo continuo

Vamos a trabajar con un sistema realimentado, el sistema usa la señal de entrada (señal de referencia, r) y la señal de salida (señal manipulada, y) para generar una señal de error (e) a partir de la cual, por acción del sistema de control ($C(s)$), generaremos una señal de control (u) para actuar sobre el sistema ($P(s)$) y llevar la salida del sistema al valor buscado.



Si recordamos la expresión matemática del PID, el controlador que vamos a usar en esta práctica:

$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt}$$

Y en el espacio de Laplace:

$$U(s) = K_p + \frac{K_i}{s} + K_d s$$

Vemos que para *configurar* (sintonizar) un controlador PID debemos establecer el valor de tres constantes K_p , K_i y K_d .

2.1. PID con Matlab

Para definir un controlador PID en Matlab, podemos construirlo

a partir de su función de transferencia:

```
Kp = 1;  
Ki = 1;  
Kd = 1;  
  
s = tf('s');  
C = Kp + Ki/s + Kd*s;
```

o usando la función *pid*:

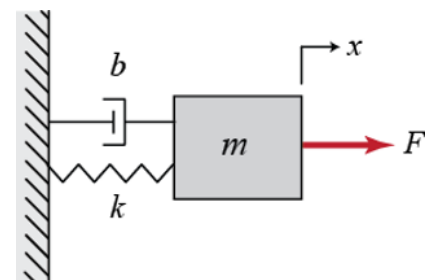
```
Kp = 1;  
Ki = 1;  
Kd = 1;  
  
C = pdi(Kp, Ki, Kd);  
tf(C) % Para mostrar la ft
```

2.2. Ejercicio: Control PID de la fuerza aplicada sobre una masa

Dado el sistema masa-resorte-amortiguador de la figura, la ecuación diferencial que modela el sistema tiene la forma:

$$m \frac{d^2 x}{dt^2} + b \frac{dx}{dt} + kx = F$$

Que en el dominio de Laplace:



$$ms^2X(s) + bsX(s) + kX(s) = F(s)$$

Es decir, que la función de transferencia del sistema es:

$$\frac{X(s)}{F(s)} = \frac{1}{ms^2 + bs + k}$$

Si los parámetros del sistema son $m = 1 \text{ kg}$, $b = 10 \text{ N s m}^{-1}$, $k = 20 \text{ N m}^{-1}$ y la fuerza aplicada es de $F = 1 \text{ N}$.

Entonces usando Matlab podemos obtener en primer lugar la respuesta en lazo abierto del sistema con:

```
s = tf('s');
P = 1/(s^2 + 10*s + 20);
step(P)
```

Para obtener la respuesta, por ejemplo, a la acción de un controlador proporcional, podemos hacer:

```
Kp = 300;
C = pid(Kp);
T = feedback(C*P, 1);
t = 0:0.01:2; % Permite definir eje de tiempos con mayor resolucion
step(T,t)
```

2.2.1. Cuestiones a resolver

Dado el planteamiento anterior para el sistema mostrado, reponder a las siguientes preguntas:

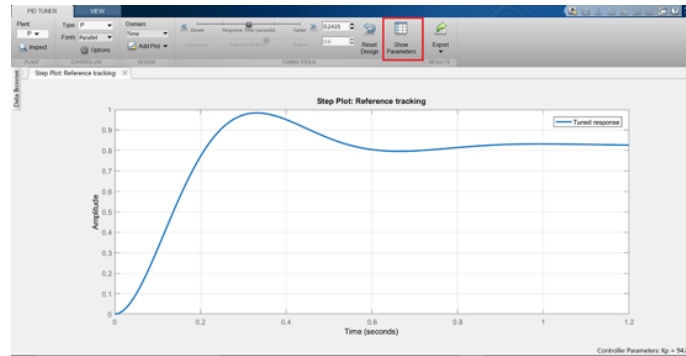
1. Obtener la respuesta en lazo abierto del sistema ante una entrada escalón unitario. Graficar la entrada y la salida del sistema. ¿De que naturaleza son las magnitudes y cuales son sus unidades de las variables en la gráfica presentada?
2. Testear la respuesta del sistema cuando se emplea un controlador P, PD, PI y PID ¿Qué diferencias se observan entre las respuestas del sistema para cada uno de los controladores? Escoger valores arbitrarios para sintonizar el PID y ajustarlos para entender el efecto que cada termino del PID.
3. Sintonizar un controlador PID de forma manual, intentando obtener el menor tiempo de subida posible sin que se produzca sobreoscilación o sobrepasamiento. El tiempo de subida ha de ser inferior a 0,1.
4. Desarrollar en Matlab un script que obtenga de forma automática los parámetros de controladores P, PI y PID, empleando el método de Ziegler-Nichols de lazo abierto, descrito en la teoría.
5. Comparar la respuesta entre el sistema controlador por el PID sintonizado manualmente y el sintonizado mediante Ziegler-Nichols

2.3. Sintonizado de PID automático (Autotune)

Matlab ofrece una herramienta automática para escoger el PID accesible a través del comando *pidTuner*. Para obtener el controlador proporcional óptimo (según este autotuner) para el proceso P anteriormente definido

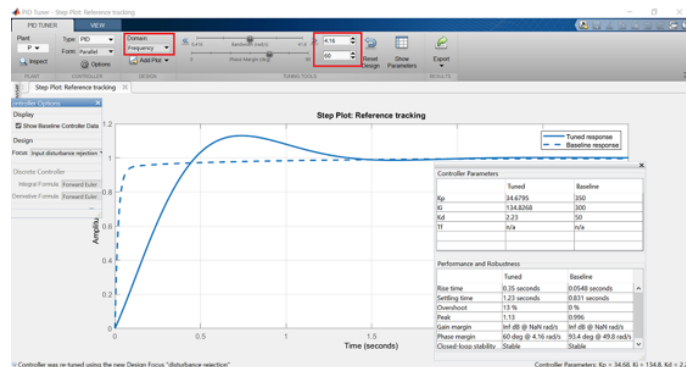
```
pidTuner(P, 'p');
```

Hacer click en "Show Parameters" para ver los valores de sintonización del controlador.



También podemos realizar la comparación entre un PID base y un PID optimizado, pasando como segundo parámetro a la función *pidTuner* la función de transferencia de otro PID (baseline).

```
pidTuner(P, 'p');
```



De nuevo, haciendo clic en "Show Parameters" se pueden comparar las constantes y características dinámicas del sistema controlado por un PID sintonizado automáticamente y otro previo (baseline).

Por último, mediante los controles disponibles en la herramienta (indicados en la figura anterior), se pueden ajustar los parámetros dinámicos para ajustar la respuesta del sistema.

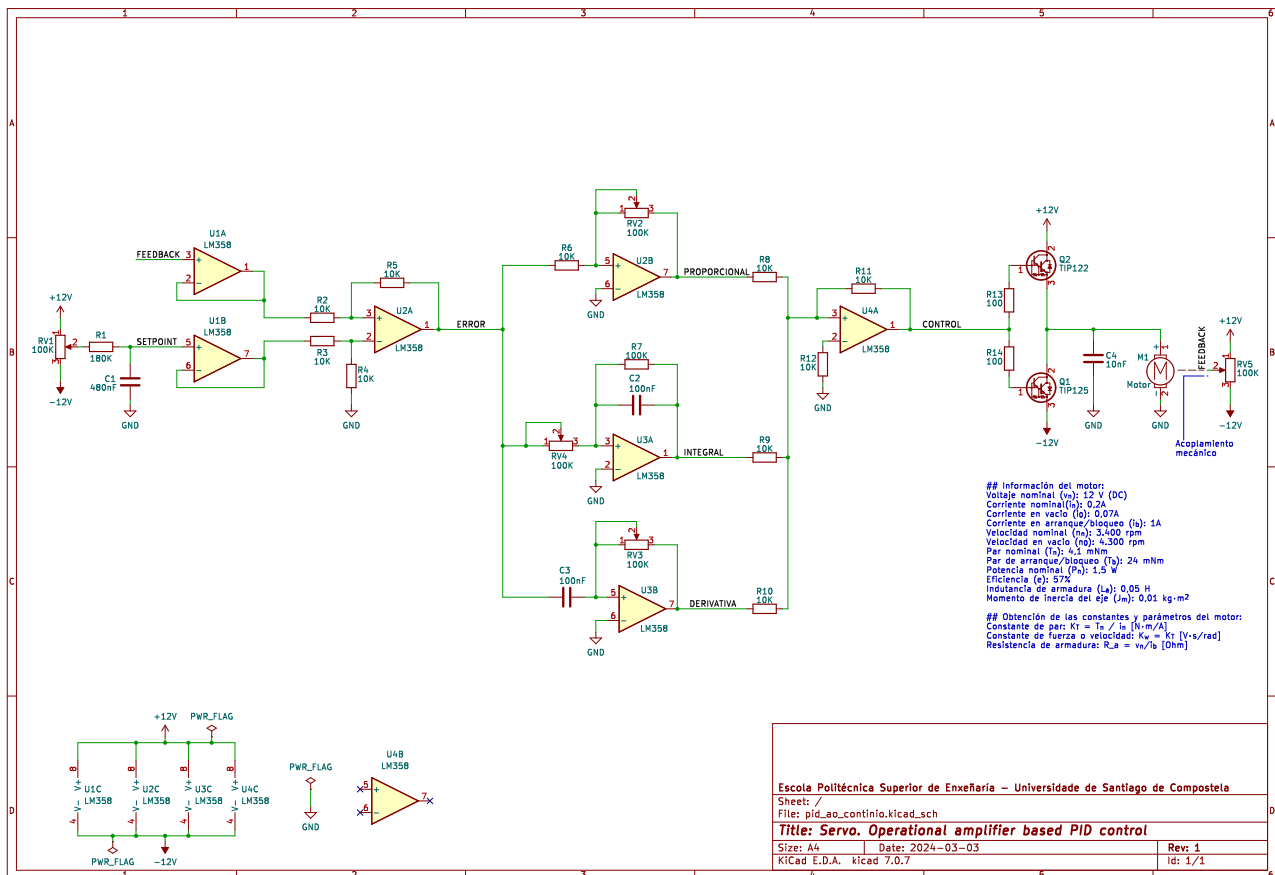
2.3.1. Cuestiones

1. Obtener las respuestas del sistema cuando está controlado con un controlador P, PD, PI y PID sintonizados automáticamente con *pidTuner*.
2. Comparar las respuestas obtenidas para cada tipo de controlador, ¿que diferencias se observan?
3. Comparar las respuestas obtenidas con los controladores sintonizados con *pidTuner* y los sintonizados usando el método de Ziegler Nichols.

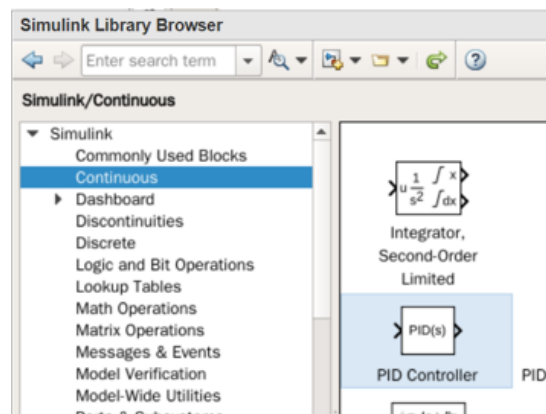
- Comparar el resultado de la sintonización de automática PID para el sistema inicial (función de transferencia P) con el PID sintonizado manualmente para minimizar el tiempo de subida. Usando los controles de *pidTuner* para ajustar los parámetros dinámicos para igualar

2.4. PID con Simulink

Implementar el sistema (servomotor) de la figura, empleando elementos básicos de Simulink. Suponer unos valores arbitrarios para las constantes del PID.



Sustituir el PID con realizado con amplificadores operacionales con el bloque PID disponible en Simulink



El motor empleado tiene las siguientes características:

Parameter	Units	Value	Parameter	Units	Value
Nominal voltaje	V	12	Max power	W	1.5
No-load speed	rpm	4300	Max efficiency	%	57
No-load current	A	0.07	Terminal resistance	Ω	12
Rated speed	rpm	3400	Terminal inductance	H	0.05
Rated torque	$\text{mN} \cdot \text{m}$	4.1	Torque constant	$\text{mN} \cdot \text{m/A}$	20.5
Stall torque	$\text{mN} \cdot \text{m}$	24	Speed constant	$\text{V} \cdot \text{s/rad}$	167
Starting current	A	1.0	Axis inertia	$\text{kg} \cdot \text{m}^2$	0.01

Además tiene acoplado una reductora de relación 48:1 en su eje.

2.4.1. Cuestiones

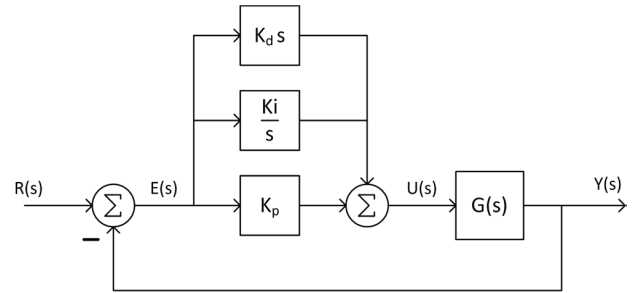
1. Como usarías el método de Ziegler-Nichols para sintonizar las constantes del PID
2. Prueba las herramientas de Autotune de las de dispone el bloque PID de Simulink.

Práctica 3. Compensación Proporcional-Integral-Derivativa

La compensación proporcional-integral-derivativa combina la acción de la compensación PI y PD:

1. La compensación PD ajusta la respuesta transitoria.
2. La compensación PI mejora el error en estado estacionario.

La función de transferencia de un controlador PID, visto en teoría es:



$$D(s) = K_p + \frac{K_i}{s} + K_d s = \frac{K_d s^2 + K_p s + K_i}{s}$$

Es decir, la función de transferencia del controlador PID tiene dos ceros y un polo en el origen:

- La pareja polo/cero cerca del origen se determinan mediante lo visto para el diseño de un compensador PI.
- La localización del segundo cero se determina mediante lo visto para el compensador PD.

Procedimiento de diseño de un compensador PID

1. Determinar la posición de los polos dominantes a partir de los requerimientos de diseño para la respuesta en lazo cerrado.
2. Diseñar un compensador PD (posición del cero y ganancia) para colocar el lugar de las raíces adecuado y obtener los polos en lazo cerrado buscados.
3. Simular el sistema con el compensador PD e iterar si es necesario para terminar de ajustar la respuesta del sistema.
4. Diseñar el compensador PI y añadirlo al sistema ya compensado con PD, determinar la ganancia necesaria para mantener la localización de los polos dominantes en lazo cerrado buscados.
5. Determinar los parámetros del controlador PID: K_p , K_i y K_d .
6. Simular el sistema PID-compensado e iterar para terminar de ajustar la respuesta.

3.1. Ejercicio: Sistema con compensación PID

Diseñar la compensación PID para satisfacer las siguientes especificaciones:

- $t_s = 2s$. Considerando un error de 2 %.
- $\%OS = 20\%$
- Error en estado estacionario cero para una referencia constante.

Para un sistema cuya función de transferencia es:

$$F(s) = \frac{15}{s^3 + 10s^2 + 27s + 18}$$

3.1.1. Cuestiones

1. Obtener los polos dominantes buscados, en lazo cerrado para las especificaciones listadas anteriormente.
2. Obtener el lugar de las raíces para este sistema en lazo cerrado con un controlador proporcional.
3. Diseñar un compensador PD que satisfaga las condiciones dinámicas:
 - a) Calcular la contribución angular del cero del compensador PD para que se cumpla el criterio de ángulos.
 - b) Obtener la posición del cero en el eje real.
 - c) Obtener el valor de ganancia necesario para conseguir los polos buscado, aplicando el criterio de magnitud.
 - d) Simular la respuesta del sistema ante una entrada escalón y comparar la respuesta del sistema compensado y sin compensar (con controlador proporcional).
4. Añadir un compensador PI, elegir la posición de su cero y simular e iterar hasta obtener la respuesta buscada.
5. Obtener los valores de las constantes del PID diseñado.

Práctica 4. PID en tiempo discreto

El análisis de sistemas de control en tiempo discreto es muy similar al análisis en tiempo continuo. La principal diferencia es que en tiempo discreto, las señales y las funciones de transferencia se representan en el dominio de la frecuencia de la transformada Z en lugar de la transformada de Laplace.

Para realizar el diseño de la implementación de un controlador PID en un microcontrolador (MCU) o una FPGA, se requiere discretizar el controlador PID. La discretización de un controlador PID. Para esto se realizan aproximaciones para las derivadas de primer orden empleando diferencias finitas.

La señal de salida del controlador y la señal de error se discretizan en el tiempo con un periodo de muestreo Δt .

4.1. Discretización de un controlador PID

La ecuación diferencial de un controlador PID en tiempo continuo es:

$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt} \quad (4.1)$$

y derivando ambos miembros de la ecuación respecto al tiempo, obtenemos:

$$\frac{du(t)}{dt} = K_p \frac{de(t)}{dt} + K_i e(t) + K_d \frac{d^2 e(t)}{dt^2} \quad (4.2)$$

Si la derivada se aproxima como:

$$\frac{f(t)}{dt} \approx \frac{f(t_k) - f(t_{k-1})}{\Delta t} \quad (4.3)$$

siendo k el número de muestra y ΔT el periodo de muestreo.

Entonces la ecuación diferencial del controlador PID se puede aproximar como:

$$\frac{u_k - u_{k-1}}{\Delta t} = K_p \frac{e_k - e_{k-1}}{\Delta t} + K_i e_k + K_d \frac{e_k - 2e_{k-1} + e_{k-2}}{\Delta t^2} \quad (4.4)$$

que simplificando y agrupando términos tenemos la ecuación para un PID en tiempo discreto:

$$u_k = u_{k-1} + \left(K_p + K_i \Delta t + \frac{K_d}{\Delta t} \right) e_k - \left(K_p + \frac{2K_d}{\Delta t} \right) e_{k-1} + \frac{K_d}{\Delta t} e_{k-2} \quad (4.5)$$

o, en términos de los tiempos de integración y derivación:

$$u_k = u_{k-1} + K_p \left[\left(1 + \frac{\Delta t}{T_i} + \frac{T_d}{\Delta t} \right) e_k - \left(1 + \frac{2T_d}{\Delta t} \right) e_{k-1} + \frac{T_d}{\Delta t} e_{k-2} \right] \quad (4.6)$$

donde $T_i = K_i/K_p$ y $T_d = K_d/K_p$.

4.1.1. Implementación de controlador PID en tiempo discreto

Las ecuaciones anteriores se pueden implementar en un microcontrolador o una FPGA. La implementación mediante un bucle de control que se ejecuta a la frecuencia de muestreo Δt .

El siguiente pseudocódigo muestra la implementación de un controlador PID en tiempo discreto:

```
previous_error := 0
integral := 0
start:
    error := setpoint - measured_value
    integral := integral + error * dt
    derivative := (error - previous_error) / dt
    output := Kp * error + Ki * integral + Kd * derivative
    previous_error := error
    wait(dt)
    goto start
```

4.1.2. PID como un filtro IIR

La transformada Z de un controlador PID en tiempo discreto es:

$$C(z) = K_p + \frac{K_i \Delta t}{z - 1} + \frac{K_d z - 1}{\Delta t z} \quad (4.7)$$

y expresada en forma de IIR:

$$C(z) = \frac{(K_p + K_i \Delta t + \frac{K_d}{\Delta t}) + (-K_p - \frac{2K_d}{\Delta t}) z^{-1} + \frac{K_d}{\Delta t} z^{-2}}{1 - z^{-1}} \quad (4.8)$$

de la cual se puede deducir el bucle de control implementado habitualmente en PLCs y FPGAs.

```
A0 := Kp + Ki * dt + Kd / dt
A1 := -Kp - 2 * Kd / dt
A2 := Kd / dt
error[2] := 0
error[1] := 0
error[0] := 0
output := u0 // Normalmente el valor inicial del actuador
start:
    error[2] := error[1]
    error[1] := error[0]
    error[0] := setpoint - measured_value
    output := A0 * error[0] + A1 * error[1] + A2 * error[2]
    wait(dt)
    goto start
```

4.1.3. Filtro pasa baja en el término derivativo

El término derivativo de un controlador PID en tiempo discreto puede causar problemas de ruido en la señal de control, debido a cambios bruscos de la señal de error (flancos de subida o bajada). Para evitar estos problemas, se puede añadir un filtro pasa baja a la acción derivativa de constante de tiempo $\tau = T_d/N$, con $3 \leq N \leq 10$, cuya expresión es:

$$C(z) = \frac{1}{\tau z^{-1} + 1} \quad (4.9)$$

y la ecuación del controlador PID con filtro pasa baja es:

$$C(z) = K_p + \frac{K_i \Delta t}{z - 1} + \frac{K_d T_d}{\Delta t N} \frac{z^{-1}}{z^{-1} + 1} \quad (4.10)$$

El pseudocódigo que implementa este tipo de controlador PID, con filtro pasa baja, es:

```
A0 := Kp + Ki*dt
A1 := -Kp
error[2] := 0 // e(t-2)
error[1] := 0 // e(t-1)
error[0] := 0 // e(t)
output := u0 // Normalmente el valor inicial del actuador
A0d := Kd/dt
A1d := - 2.0*Kd/dt
A2d := Kd/dt
N := 5
tau := Kd / (Kp*N) // Constante de tiempo del filtro pasa baja
alpha := dt / (2*tau)
alpha_1 = alpha / (alpha + 1)
alpha_2 = (alpha - 1) / (alpha + 1)
d0 := 0
d1 := 0
fd0 := 0
fd1 := 0
loop:
    error[2] := error[1]
    error[1] := error[0]
    error[0] := setpoint - measured_value
    // PI
    output := output + A0 * error[0] + A1 * error[1]
    // Filtered D
    d1 := d0
    d0 := A0d * error[0] + A1d * error[1] + A2d * error[2]
    fd1 := fd0
    fd0 := alpha_1 * (d0 + d1) - alpha_2 * fd1
    output := output + fd0
    wait(dt)
    goto loop
```

4.2. Tareas a realizar

- Implementar los tres pseudocódigos anteriores en el microcontrolador ESP32, empleado para la realización del proyecto del péndulo invertido (ver capítulo 6).
- Integrar los códigos realizados en una biblioteca de control PID de vuestra realización. A modo de guía para crear la biblioteca, se puede seguir el tutorial de la realización de una biblioteca en C++ para Arduino en <https://www.arduino.cc/en/Hacking/libraryTutorial> o de forma más resumida en <https://sp.arduino-france.site/crear-libreria/>. En el caso de ESP32, se puede seguir el tutorial anterior si se emplea el entorno de desarrollo de Arduino.
- Realizar pruebas de funcionamiento de los controladores PID implementados en el péndulo invertido construido.
- Publicar el código de las librerías realizadas en el repositorio de GitHub del proyecto del péndulo invertido.

Práctica 5. Modelado de sistemas en el espacio de estados

Matlab proporciona un toolbox muy potente (Control System Toolbox). Este toolbox consiste en una colección de funciones que implementan técnicas para el diseño, análisis y modelado de sistemas de control lineales invariantes en el tiempo (LTI: Linear Time-Invariant). Sus características son:

- Los sistemas de control pueden modelarse como funciones de transferencia o en el espacio de estados, permitiendo el uso tanto de técnicas de espacio de estados (control moderno) como de funciones de transferencia (control clásico).
- Permite trabajar tanto con sistemas continuos como discretos.
- Se pueden realizar conversiones entre las distintas representaciones de los sistemas.
- Es posible calcular y representar respuestas en el dominio del tiempo, en el dominio de la frecuencia, así como el lugar de las raíces.
- Dispone de funciones para la asignación de polos, el control óptimo, etc.
- Dispone de una interfaz gráfica (LTI Viewer) para el análisis de la respuesta de los sistemas.

5.1. Creación del modelo de un sistema continuo en el espacio de estados

Un sistema de control LTI se modela en Matlab como un objeto. Este objeto es una estructura de datos que permite manipular el modelo del sistema como si fuera una única variable Matlab.

Para definir un sistema continuo LTI con la siguiente representación:

$$\begin{aligned}\ddot{\vec{x}}(t) &= A\vec{x}(t) + B\vec{u}(t) \\ \vec{y}(t) &= C\vec{x}(t) + D\vec{u}(t)\end{aligned}$$

podemos usar la función `ss` es decir, supongamos un sistema, con las siguientes ecuaciones de estado y salida:

$$\begin{bmatrix} \ddot{x}_1(t) \\ \ddot{x}_1(t) \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -2 & 3 \end{bmatrix} \cdot \begin{bmatrix} x_1(t) \\ x_1(t) \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} \cdot u(t)$$

$$y(t) = \begin{bmatrix} 6 & 0 \end{bmatrix} \cdot \begin{bmatrix} x_1(t) \\ x_1(t) \end{bmatrix}$$

```
A = [0 1; -2 3];  
B = [0 1]';  
C = [6 0];  
D = 0;  
sys = ss(A, B, C, D)
```

Esto ha creado un objeto llamado `sys`, correspondiente al sistema con las ecuaciones mostradas. La variable `sys` creada es una estructura de datos específica del modelo que permite manipular el sistema como una entidad única. Si quisiéramos extraer los datos asociados modelo, hay dos opciones:

1. Empleando la función `ssdata`

```
[A, B, C, D] = ssdata(sys)
```

2. Accediendo directamente a las propiedades del objeto con el modelo del sistema:

```
matriz = sys.A;
```

la variable *matriz* contiene la matriz de estado del modelo *sys*.

5.1.1. Conversión de modelos entre representación interna y externa

Para convertir un sistema entre distintas representaciones se utilizan las mismas funciones que para crear los modelos. Por ejemplo, para obtener el modelo del sistema en representación externa (*función de transferencia*) a partir de su modelo en representación interna (*ecuaciones de estado*), se ejecutaría en *Matlab*:

```
systf = tf(sys)
```

donde *sys* es el sistema modelado en el espacio de estados y *systf* el modelo en forma de función de transferencia. Para realizar la operación inversa:

```
sys = ss(systf)
```

5.1.2. Cálculo y visualización de la respuesta del sistema

El toolbox de control de Matlab también permite obtener la representación gráfica de la salida de un sistema para diferentes tipos de entradas:

- Función *step*: Representa gráficamente la respuesta del sistema ante una entrada escalón unitario, asumiendo que las condiciones iniciales son nulas.

```
step(sys)

[y, t, x] = step(sys);
% y es la respuesta del sistema
% t vector del tiempo de simulacion
% x evolucion de las variables de estado
plot(t, x)
```

- Función *initial*: Respuesta del sistema ante unas condiciones iniciales determinadas, x_0 , considerando que la entrada es nula:

```
initial(sys, x0)
% o
[y, t, x] = initial(sys, x0)
```

- Función *lsim*: Obtiene la respuesta ante una entrada arbitraria.

```
lsim(sys, u, t, x0)
% o
[y, ts, x] = lsim(sys, u, t, x0)
```

donde t es el vector de tiempo de los datos de entrada que se expresan en u . x_0 son las condiciones iniciales, si se asumen nulas no es necesario incluirlas.

El toolbox incorpora una interfaz interactiva que facilita el análisis de la respuesta de los modelos LTI, denominada LTI Viewer. Para abrir esta herramienta, ejecutar en Matlab: *ltiview*

La interfaz permite obtener la respuesta de cualquier sistema definido ante diferentes señales de entrada.

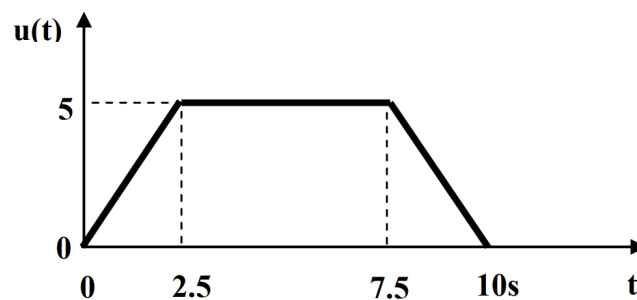
5.1.3. Cuestiones

Sea el sistema continuo definido en su representación interna por:

$$\ddot{\vec{x}}(t) = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -50 & -20 & -10 \end{bmatrix} \cdot \vec{x}(t) + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \cdot \vec{u}(t) \quad \vec{y}(t) = \begin{bmatrix} 30 & 0 & 0 \end{bmatrix} \cdot \vec{x}(t)$$

Utilizando las funciones necesarias:

1. Representar el sistema como un objeto LTI.
2. Obtener la respuesta del sistema, la evolución de las variables de estado, considerando una entrada escalón unitario y condiciones iniciales nulas.
3. Obtener la respuesta del sistema y la evolución de las variables de estado, considerando condiciones iniciales $x_0 = [0, 5, -1]$ y entrada nula.
4. Obtener la respuesta del sistema y evolución de los estados ante una entrada escalón unitario y condiciones iniciales $x_0 = [0, 5, -1]$.
5. Obtener la respuesta del sistema cuando la entrada es como la que se muestra en la figura y suponiendo condiciones iniciales nulas



6. Obtener la función de transferencia del sistema y la salida de dicha función ante un escalón unitario, comprobar que el resultado coincide con la 2ª cuestión.

5.2. Controlabilidad y observabilidad de un sistema

En Matlab podemos evaluar la controlabilidad del estado de un sistema usando la función `ctrb`, que devuelve la matriz de controlabilidad del sistema y evaluando su rango con la función `rank` podemos determinar si un sistema es o no controlable.

Del mismo modo con la función `obsv` se obtiene la matriz de observabilidad y con ella podremos evaluar si el estado de un sistema es o no observable a partir de un conjunto de salidas.

5.2.1. Cuestiones

Usando la descripción del sistema del apartado anterior

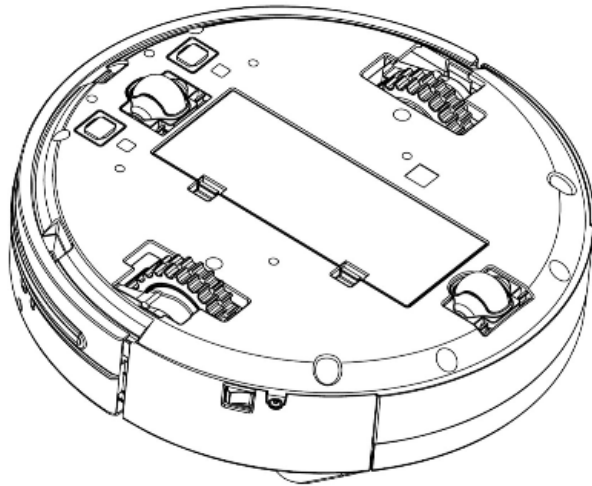
1. Obtener la matriz de observabilidad y de controlabilidad del sistema:

- a) Empleando las funciones *ctrlb* y *obsv*.
- b) Realizando el cálculo sin emplear las funciones anteriores, solo operaciones básicas con matrices.

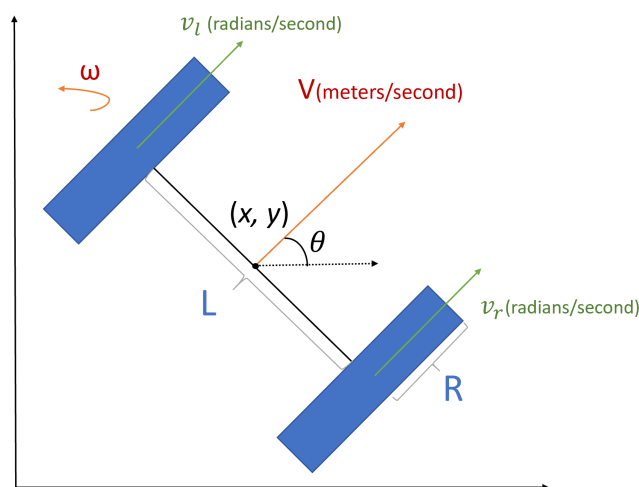
Comprobar que el resultado es el mismo por ambos métodos.

2. Evaluar si el sistema es observable y controlable.

5.3. Modelo de estados del robot Turtlebot (opcional)



Un robot con ruedas es aquel que funciona en contacto con el suelo y su movimiento se basa en ruedas accionadas por separado situadas a ambos lados del cuerpo del robot. Así, puede cambiar de dirección variando la velocidad relativa de rotación de sus ruedas. En general, independientemente del número de ruedas que tenga, la velocidad de cada lado tendrá el mismo valor por separado. Por lo tanto, todos los robots con ruedas pueden ser tratados como robots de tracción diferencial con dos ruedas.



Si se está diseñando un robot físico y se necesita conocer las verdaderas entradas a las ruedas, es necesario modelar un robot de tracción diferencial. El diferencial de tracción es un sistema de tracción de dos ruedas con actuadores independientes. Normalmente se colocan dos ruedas motrices a cada lado del robot y la dirección hacia el frente se trata como la dirección positiva. Tendrá un estado

definido por las variables (x, y, θ) , que será la posición y dirección del robot en todo instante, y las entradas de este sistema mecánico serán las velocidades de la rueda izquierda (v_l) y derecha (v_r).

Ref: <https://ucr-robotics.readthedocs.io/en/latest/tbot/moRbt.html>

5.4. Cuestiones

1. Teniendo en cuenta lo anteriormente descrito, plantea un modelo de estados para el movimiento del Turtlebot, usando como variables de entrada y salida las descritas anteriormente. Tener en cuenta que la distancia entre ruedas es $L = 23\text{cm}$ y el diametro de las ruedas es de $R = 3,5\text{ cm}$
2. Teniendo en cuenta la información disponible para las características de los motores que accionan las ruedas del Turtlebot, construye un modelo de estados para el sistema electromecánico de accionamiento. Las características del modelo de motor en el Turtlebot, están en “*Kobuki User Guide*” disponible en el anexo [B](#).

Práctica 6. Proyecto péndulo invertido

El objetivo de esta tarea es que durante la duración de la materia realices la construcción de un péndulo invertido y su control. El péndulo invertido es un sistema no lineal, inestable y de doble integración. La inestabilidad del sistema hace que sea un problema interesante para el control, ya que el controlador debe ser capaz de estabilizar el sistema y mantenerlo en una posición vertical.

6.1. Objetivos

- Elegir y adquirir componentes electromecánicos y electrónicos para el proyecto.
- Diseñar e imprimir/fabricar el sistema mecánico del péndulo invertido mediante impresión 3D, corte laser u otra técnica adecuada.
- Diseñar e implementar el controlador del péndulo invertido. Diseño de PCB, fabricación y soldadura de componentes.
- Realizar la implementación del software de control en el microcontrolador ESP32 o Arduino nano.
- Sintonizar y ajustar el control PID para el péndulo invertido.
- Cumplir las especificaciones a continuación para el sistema completo.

6.2. Especificaciones

- El sistema debe ser capaz de mantenerse en posición vertical con una tolerancia de $\pm 5^\circ$.
- El sistema debe compensar los empujones externos y volver a la posición vertical.
- El sistema debe ser capaz de poder iniciar su funcionamiento desde cualquier posición.

6.3. Entregables

- Repositorio en GitHub con la descripción del sistema, diseño y elementos necesarios para la fabricación de la estructura, diseños y elementos para fabricación de la PCB, implementación del software y sintonización del controlador.
- El repositorio debe contener un informe en formato markdown en el fichero README.md del propio repositorio, siguiendo la línea del ejemplo mostrado a continuación.
- Demostración del sistema funcionando en una fecha anterior al final de la materia.

6.4. Aclaraciones

- Para la realización del diseño de partes mecánicas del péndulo invertido, se deberá un de software de diseño 3D “profesional”, como puede ser Fusion 360, FreeCAD, Catia, SolidWorks,

etc. No se aceptarán diseños realizados en Tinkercad o similares. Se puede acceder a cuentas gratuitas de estudiantes para Fusion 360 (y el resto del software de Autodesk) en <https://www.autodesk.com/education/home>, registrándose con el correo de la USC. Todos los proyectos deben incluir algún componente mecánico de diseño y fabricación propia.

- Para la fabricación de las piezas mecánicas, se puede usar cualquier técnica de fabricación, como por ejemplo impresión 3D, corte laser, etc. La escuela dispone de un taller de fabricación con impresoras 3D y cortadoras laser y otras herramientas: Laboratorio de Instrumentación. Para poder acceder solicitarlo al profesor de la materia.
- Para realizar el diseño de la PCB (esquemático, pcb y ficheros de fabricación) utilizar el software de diseño de PCBs [KiCad](#). No se aceptarán montajes con elementos electrónicos sueltos o montados en protoboard.
- Para la fabricación de la PCB, se puede usar cualquier servicio de fabricación de PCBs, como por ejemplo [JLCPCB](#), [PCBWay](#), etc. Existe una web donde se pueden comparar los precios de diferentes fabricantes de PCBs: <https://pcbshopper.com/>.

6.5. Ejemplo de informe

Para que os sirva de guía a la hora de realizar el proyecto y preparar una documentación, os adjunto, en el anexo [A](#), un ejemplo de informe de la realización hecha por Daniel Acevedo de este proyecto de péndulo invertido.

Anexo A. Ejemplo de informe del proyecto péndulo invertido

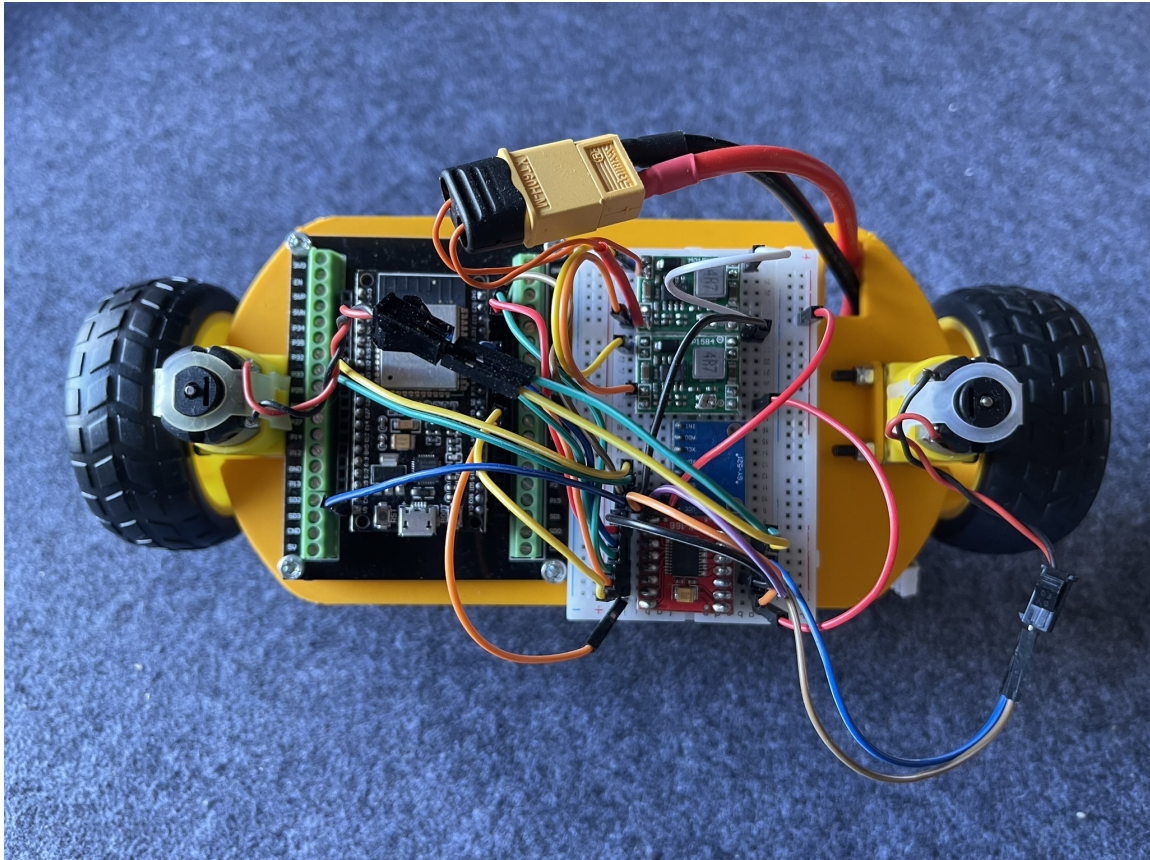
Ejemplo de informe sobre el proyecto/práctica a realizar sobre el péndulo invertido. Este documento debe servir de guía tanto para la realización del proyecto como para la preparación de la documentación a anexar en el repositorio de GitHub del proyecto realizado.

Siendo una guía, se tendrá en cuenta que cada proyecto es diferente y por lo tanto, la documentación a entregar también lo será. Por lo tanto, este ejemplo de informe debe ser tomado como una guía y no como un documento a copiar y entregar.

Péndulo Invertido

Daniel Acevedo Facal

14 de diciembre de 2023



1. Introducción

En este documento se recoge la creación de un primer prototipo de un robot tipo "péndulo invertido".

2. Metodología seguida

1. **Buscar proyectos similares:**
Investigar si ya existe un proyecto de un robot similar que esté funcional y documentado.
2. **Investigación general:**
Como no existe ningún proyecto bien hecho y bien documentado, procedemos a investigar cómo podemos llevar a cabo el proyecto desde cero.
3. **Fijación de objetivos:**
Es importante tener metas para saber qué hardware necesitamos.
4. **Selección del Hardware:**
Una vez que sabemos qué queremos lograr con nuestro robot, tenemos que buscar el hardware con las características necesarias.
5. **Diseño del Chasis:**
En este apartado debemos resolver dos problemáticas: tener el peso compensado y tener el centro de gravedad bajo.
6. **Prueba de componentes por separado:**
Verificamos que todo funcione por separado para luego juntar todo.
7. **Montaje:** Montamos todos los componentes en el chasis para poder probar el robot al completo.
8. **Programación inicial:**
Hacemos un primer código que solo mantenga el robot estático en un mismo lugar.
9. **Ajuste del PID:**
Ajustamos los valores del PID para que el robot se mantenga estático.
10. **Implementación de control por mando:**
Realizamos un código que nos permita desplazar el robot con un mando Bluetooth.

3. Objetivos

1. **Estabilidad Perpendicular al Suelo:** Garantizar que el robot sea capaz de mantenerse estable en posición perpendicular al suelo. **Logrado**
2. **Compensación de Empujones:**
Desarrollar la capacidad del robot para compensar empujones y evitar caídas. **Logrado**
3. **Arranque desde Cualquier Posición:**
Lograr que el robot pueda iniciar su funcionamiento desde cualquier posición inicial. **Logrado**
4. **Reducción de Posibilidades de Volcar:**
Minimizar las posibilidades de volcamiento y asegurar que el robot no pueda volver a la posición incorrecta. **Logrado**
5. **Control mediante Mando Bluetooth:**
Implementar un sistema de control que permita manejar el robot a través de un mando Bluetooth.

4. Investigación

En relación con la búsqueda de otros proyectos de este estilo, he detectado una problemática: hay varios proyectos hechos con el mismo principio, pero nadie ha creado una documentación del proyecto para ayudar a otras personas que quieran hacerlo. Por lo tanto, tuve que empezar casi desde cero.

Aunque hay esta escasez de información, he podido rescatar algo de información examinando fotos y vídeos. De ellos, he determinado el hardware que se necesita y el tipo de control para manejar el movimiento (PID).

5. Hardware

Para la selección del hardware, se buscó que el robot se hiciera con elementos básicos y comunes para que sea un robot de bajo costo.

1. **ESP32:**

Para el microcontrolador, he elegido una ESP32, la cual me permitirá conectar un mando a ella mediante Bluetooth e incluso se podría usar su conectividad WiFi para diversas funciones. En concreto, se ha utilizado una ESP32-devkitc-v4 que ronda los 5€.

He consultado un par de páginas web para informarme al respecto de esta placa, para no alargar este documento dejo los enlaces a continuación: [enlace01](#), [enlace02](#).

2. **MPU-6050:**

Este sensor tiene incorporado un acelerómetro y un giroscopio, los cuales vamos a usar para medir la orientación del robot. Las fórmulas e información necesaria que he necesitado para usar este sensor están en [esta página web](#).

3. **TB6612FNG:**

El driver que he elegido es una solución más moderna que el común puente H; su precio es de menos de 2€ y nos permite controlar 2 motores simultáneamente y de manera independiente. Al igual que con el anterior sensor, la página de la que extraje toda la información la dejo en el siguiente [enlace](#).

4. **Regulador de tensión:**

Pendiente de documentar lo que se usó en este aspecto.

5. **Batería:**

Primero se realizó el robot con un portapilas de 4 pilas AA, pero posteriormente se ha implementado una batería LiPo que nos proporciona más autonomía y facilidad a la hora de recargar el robot.

6. **Motores y ruedas:**

En este proyecto se han usado los comúnmente usados motores de DC con una reductora, la cual permite un fácil acople de una rueda que normalmente se vende con el propio motor.

6. Chasis

He diseñado 4 chasis para el robot, y con el tercero he logrado hacer funcionar el robot. En el desarrollo de estos 3 chasis, he podido valorar cuáles son los factores determinantes para que el robot funcione.

1. Centro de gravedad:

Es determinante que el centro de gravedad del robot sea el más bajo posible. Si el peso está muy elevado respecto al suelo, se hará prácticamente imposible que el robot sea capaz de mantenerse derecho.

2. Peso centrado:

Al igual que el centro de gravedad bajo, es decisivo que el peso del robot esté centrado en el eje de las ruedas, lo que hará que, combinado con el apartado anterior, el robot debería mantenerse en la posición que queremos sin necesidad de que el robot esté encendido. Cuanto más estable sea el robot gracias al diseño del chasis, más sencillo nos será ajustar el PID hasta lograr que el robot funcione correctamente.

Ahora voy a explicar los problemas de mis distintos chasis hasta el momento en que fui capaz de hacer que el robot funcionara.

1. Primer chasis:

Este estaba bastante mal hecho, ya que el centro de gravedad estaba demasiado alto, lo que hacía muy complicado que el robot fuese capaz de mantenerse en posición vertical.

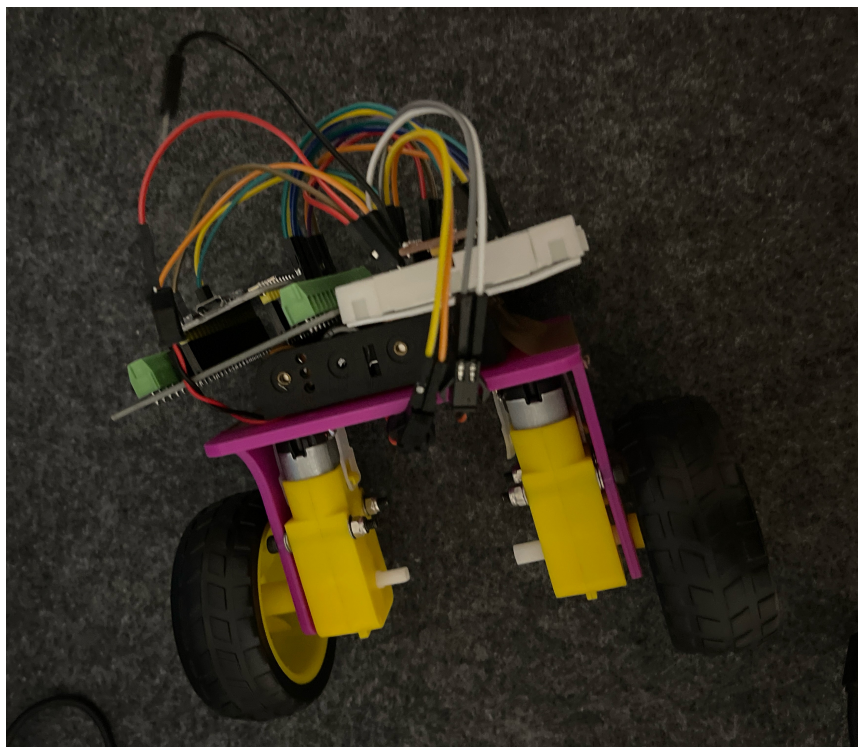


Figura 1: Chasis01

2. Segundo chasis:

Aunque el centro de gravedad era más bajo en este segundo chasis, el problema fue que el peso no estaba centrado, y el robot tendía a inclinarse hacia un lado constantemente.

3. Tercer chasis:

Después de corregir los errores de los dos chasis anteriores, logré que el robot funcionara. En este

caso, el centro de gravedad estaba muy centrado y a una altura baja, lo que proporcionaba una gran estabilidad incluso cuando el robot estaba apagado.

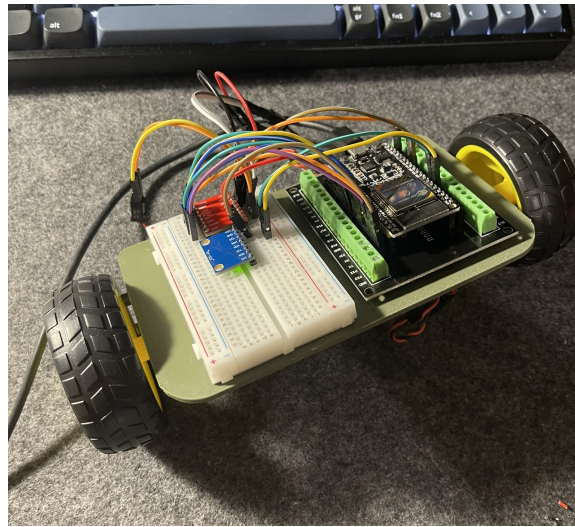


Figura 2: Chasis03

4. Cuarto chasis:

Este chasis (foto de la portada) me permite incorporar una batería LiPo sin dejar de tener en cuenta las características del anterior, que ya funcionaba con pilas.



Figura 3: Chasis04 parte inferior



Figura 4: Chasis04 parte superior

7. Prueba de componentes por separado

■ MPU-6050:

Realizamos una primera prueba para verificar que el sensor funcione.

```

1 // Librerias I2C para controlar el mpu6050
2 // la libreria MPU6050.h necesita I2Cdev.h, I2Cdev.h necesita Wire.h
3 #include "I2Cdev.h"
4 #include "MPU6050.h"
5 #include "Wire.h"
6
7 // La direcci n del MPU6050 puede ser 0x68 o 0x69, dependiendo
8 // del estado de AD0. Si no se especifica, 0x68 estar implicito
9 MPU6050 sensor;
10
11 // Valores RAW (sin procesar) del acelerometro y giroscopio en los ejes x,y,z
12 int ax, ay, az;
13 int gx, gy, gz;
14
15 void setup() {
16   Serial.begin(57600);    //Iniciando puerto serial
17   Wire.begin();          //Iniciando I2C
18   sensor.initialize();    //Iniciando el sensor
19
20   if (sensor.testConnection()) Serial.println("Sensor iniciado correctamente");
21   else Serial.println("Error al iniciar el sensor");
22 }
23
24 void loop() {
25   // Leer las aceleraciones y velocidades angulares
26   sensor.getAcceleration(&ax, &ay, &az);
27   sensor.getRotation(&gx, &gy, &gz);
28
29   //Mostrar las lecturas separadas por un [tab]
30   Serial.print("a[x y z] g[x y z]:\t");

```

```

31 Serial.print(ax); Serial.print("\t");
32 Serial.print(ay); Serial.print("\t");
33 Serial.print(az); Serial.print("\t");
34 Serial.print(gx); Serial.print("\t");
35 Serial.print(gy); Serial.print("\t");
36 Serial.println(gz);
37
38 delay(100);
39 }

```

Listing 1: Código para probar el MPU6050

Como vemos que el sensor nos proporciona medidas coherentes, procedemos a calibrar el sensor en la posición en la que estará midiendo.

```

1 // Librerías I2C para controlar el mpu6050
2 // la librería MPU6050.h necesita I2Cdev.h, I2Cdev.h necesita Wire.h
3 #include "I2Cdev.h"
4 #include "MPU6050.h"
5 #include "Wire.h"
6
7 // La dirección del MPU6050 puede ser 0x68 o 0x69, dependiendo
8 // del estado de AD0. Si no se especifica, 0x68 estará implícito
9 MPU6050 sensor;
10
11 // Valores RAW (sin procesar) del acelerómetro y giroscopio en los ejes x,y,z
12 int ax, ay, az;
13 int gx, gy, gz;
14
15 //Variables usadas por el filtro pasa bajos
16 long f_ax,f_ay, f_az;
17 int p_ax, p_ay, p_az;
18 long f_gx,f_gy, f_gz;
19 int p_gx, p_gy, p_gz;
20 int counter=0;
21
22 //Valor de los offsets
23 int ax_o,ay_o,az_o;
24 int gx_o,gy_o,gz_o;
25
26 void setup() {
27   Serial.begin(57600); //Iniciando puerto serial
28   Wire.begin(); //Iniciando I2C
29   sensor.initialize(); //Iniciando el sensor
30
31   if (sensor.testConnection()) Serial.println("Sensor iniciado correctamente");
32
33   // Leer los offset los offsets anteriores
34   ax_o=sensor.getXAccelOffset();
35   ay_o=sensor.getYAccelOffset();
36   az_o=sensor.getZAccelOffset();
37   gx_o=sensor.getXGyroOffset();
38   gy_o=sensor.getYGyroOffset();
39   gz_o=sensor.getZGyroOffset();
40
41   Serial.println("Offsets:");
42   Serial.print(ax_o); Serial.print("\t");
43   Serial.print(ay_o); Serial.print("\t");
44   Serial.print(az_o); Serial.print("\t");
45   Serial.print(gx_o); Serial.print("\t");
46   Serial.print(gy_o); Serial.print("\t");
47   Serial.print(gz_o); Serial.print("\t");
48   Serial.println("\nnEnvíe cualquier carácter para empezar la calibraciónnn");
49   // Espera un carácter para empezar a calibrar
50   while (true){if (Serial.available()) break;}
51   Serial.println("Calibrando, no mover IMU");
52
53 }
54
55 void loop() {
56   // Leer las aceleraciones y velocidades angulares
57   sensor.getAcceleration(&ax, &ay, &az);

```

```

58     sensor.getRotation(&gx, &gy, &gz);
59
60     // Filtrar las lecturas
61     f_ax = f_ax-(f_ax>>5)+ax;
62     p_ax = f_ax>>5;
63
64     f_ay = f_ay-(f_ay>>5)+ay;
65     p_ay = f_ay>>5;
66
67     f_az = f_az-(f_az>>5)+az;
68     p_az = f_az>>5;
69
70     f_gx = f_gx-(f_gx>>3)+gx;
71     p_gx = f_gx>>3;
72
73     f_gy = f_gy-(f_gy>>3)+gy;
74     p_gy = f_gy>>3;
75
76     f_gz = f_gz-(f_gz>>3)+gz;
77     p_gz = f_gz>>3;
78
79     //Cada 100 lecturas corregir el offset
80     if (counter==100){
81         //Mostrar las lecturas separadas por un [tab]
82         Serial.print("promedio:"); Serial.print("\t");
83         Serial.print(p_ax); Serial.print("\t");
84         Serial.print(p_ay); Serial.print("\t");
85         Serial.print(p_az); Serial.print("\t");
86         Serial.print(p_gx); Serial.print("\t");
87         Serial.print(p_gy); Serial.print("\t");
88         Serial.println(p_gz);
89
90         //Calibrar el acelerometro a 1g en el eje z (ajustar el offset)
91         if (p_ax>0) ax_o--;
92         else {ax_o++;}
93         if (p_ay>0) ay_o--;
94         else {ay_o++;}
95         if (p_az-16384>0) az_o--;
96         else {az_o++;}
97
98         sensor.setXAccelOffset(ax_o);
99         sensor.setYAccelOffset(ay_o);
100        sensor.setZAccelOffset(az_o);
101
102        //Calibrar el giroscopio a 0 /s en todos los ejes (ajustar el offset)
103        if (p_gx>0) gx_o--;
104        else {gx_o++;}
105        if (p_gy>0) gy_o--;
106        else {gy_o++;}
107        if (p_gz>0) gz_o--;
108        else {gz_o++;}
109
110        sensor.setXGyroOffset(gx_o);
111        sensor.setYGyroOffset(gy_o);
112        sensor.setZGyroOffset(gz_o);
113
114        counter=0;
115    }
116    counter++;
117 }

```

Listing 2: Código para calibrar el MPU6050

Finalmente, probamos el código que utilizaremos para medir el ángulo de inclinación del robot.

```

1 // Librerias I2C para controlar el mpu6050
2 // la libreria MPU6050.h necesita I2Cdev.h, I2Cdev.h necesita Wire.h
3 #include "I2Cdev.h"
4 #include "MPU6050.h"
5 #include "Wire.h"
6
7 // La direcci n del MPU6050 puede ser 0x68 o 0x69, dependiendo

```



```

8 // del estado de ADO. Si no se especifica, 0x68 estar implicito
9 MPU6050 sensor;
10
11 // Valores RAW (sin procesar) del acelerometro y giroscopio en los ejes x,y,z
12 int ax, ay, az;
13 int gx, gy, gz;
14 long tiempo_prev;
15 float dt;
16 float ang_x, ang_y;
17 float ang_x_prev, ang_y_prev;
18
19 void setup() {
20   Serial.begin(57600); //Iniciando puerto serial
21   Wire.begin(); //Iniciando I2C
22   sensor.initialize(); //Iniciando el sensor
23
24   if (sensor.testConnection()) Serial.println("Sensor iniciado correctamente");
25   else Serial.println("Error al iniciar el sensor");
26 }
27
28 void loop() {
29   // Leer las aceleraciones y velocidades angulares
30   sensor.getAcceleration(&ax, &ay, &az);
31   sensor.getRotation(&gx, &gy, &gz);
32
33   dt = (millis()-tiempo_prev)/1000.0;
34   tiempo_prev=millis();
35
36   //Calcular los ngulos con acelerometro
37   float accel_ang_x=atan(ay/sqrt(pow(ax,2) + pow(az,2)))*(180.0/3.14);
38   float accel_ang_y=atan(-ax/sqrt(pow(ay,2) + pow(az,2)))*(180.0/3.14);
39
40   //Calcular angulo de rotacion con giroscopio y filtro complemento
41   ang_x = 0.98*(ang_x_prev+(gx/131)*dt) + 0.02*accel_ang_x;
42   ang_y = 0.98*(ang_y_prev+(gy/131)*dt) + 0.02*accel_ang_y;
43
44
45   ang_x_prev=ang_x;
46   ang_y_prev=ang_y;
47
48   //Mostrar los angulos separadas por un [tab]
49
50   Serial.print("Rotacion en X: ");
51   Serial.print(ang_x);
52   Serial.print("\tRotacion en Y: ");
53   Serial.println(ang_y);
54
55   delay(10);
56 }

```

Listing 3: Código completo MPU 6050

Estos códigos han sido extraídos de la página web antes citada.

■ TB6612FNG:

Vamos a probar con un código básico extraído de la página web citada en la sección de hardware. Al igual que los códigos del sensor anterior, los he modificado antes de subirlos a la ESP32, ajustando los pines para que sean compatibles con la ESP32.

```

1 const int pinPWMA = 6;
2 const int pinAIN2 = 7;
3 const int pinAIN1 = 8;
4 const int pinBIN1 = 9;
5 const int pinBIN2 = 10;
6 const int pinPWMB = 11;
7 const int pinSTBY = 12;
8 const int waitTime = 2000; //espera entre fases
9 const int speed = 200; //velocidad de giro
10 const int pinMotorA[3] = { pinPWMA, pinAIN2, pinAIN1 };
11 const int pinMotorB[3] = { pinPWMB, pinBIN1, pinBIN2 };
12

```

```

13 enum moveDirection {
14     forward,
15     backward
16 };
17
18 enum turnDirection {
19     clockwise,
20     counterClockwise
21 };
22
23 void setup()
24 {
25     pinMode(pinAIN2, OUTPUT);
26     pinMode(pinAIN1, OUTPUT);
27     pinMode(pinPWMA, OUTPUT);
28     pinMode(pinBIN1, OUTPUT);
29     pinMode(pinBIN2, OUTPUT);
30     pinMode(pinPWMB, OUTPUT);
31 }
32
33 void loop()
34 {
35     enableMotors();
36     move(forward, 180);
37     delay(waitTime);
38
39     move(backward, 180);
40     delay(waitTime);
41
42     turn(clockwise, 180);
43     delay(waitTime);
44
45     turn(counterClockwise, 180);
46     delay(waitTime);
47
48     fullStop();
49     delay(waitTime);
50 }
51
52 //Funciones que controlan el vehiculo
53 void move(int direction, int speed)
54 {
55     if (direction == forward)
56     {
57         moveMotorForward(pinMotorA, speed);
58         moveMotorForward(pinMotorB, speed);
59     }
60     else
61     {
62         moveMotorBackward(pinMotorA, speed);
63         moveMotorBackward(pinMotorB, speed);
64     }
65 }
66
67 void turn(int direction, int speed)
68 {
69     if (direction == forward)
70     {
71         moveMotorForward(pinMotorA, speed);
72         moveMotorBackward(pinMotorB, speed);
73     }
74     else
75     {
76         moveMotorBackward(pinMotorA, speed);
77         moveMotorForward(pinMotorB, speed);
78     }
79 }
80
81 void fullStop()
82 {
83     disableMotors();

```

```

84   stopMotor(pinMotorA);
85   stopMotor(pinMotorB);
86 }
87
88 //Funciones que controlan los motores
89 void moveMotorForward(const int pinMotor[3], int speed)
90 {
91     digitalWrite(pinMotor[1], HIGH);
92     digitalWrite(pinMotor[2], LOW);
93
94     analogWrite(pinMotor[0], speed);
95 }
96
97 void moveMotorBackward(const int pinMotor[3], int speed)
98 {
99     digitalWrite(pinMotor[1], LOW);
100    digitalWrite(pinMotor[2], HIGH);
101
102    analogWrite(pinMotor[0], speed);
103 }
104
105 void stopMotor(const int pinMotor[3])
106 {
107     digitalWrite(pinMotor[1], LOW);
108     digitalWrite(pinMotor[2], LOW);
109
110     analogWrite(pinMotor[0], 0);
111 }
112
113 void enableMotors()
114 {
115     digitalWrite(pinSTBY, HIGH);
116 }
117
118 void disableMotors()
119 {
120     digitalWrite(pinSTBY, LOW);
121 }

```

Listing 4: Código prueba del driver y motores

8. Montaje

El montaje es muy simple; las conexiones de los sensores que debemos realizar están explicadas en las páginas web mencionadas anteriormente. Además de las conexiones de los sensores, necesitamos alimentar los motores y la ESP32. Ambos trabajan a voltajes distintos, por lo que requerimos dos reguladores de voltaje. Los motores los he alimentado a 9V, y la ESP32-DevKitC-v4 se alimenta a 5V en el pin 5V. Otros modelos se alimentan por el pin VIN a 3.3V, pero en este caso no es así. El esquema eléctrico bien hecho aún no está listo; lo añadiré a esta documentación cuando tenga la PCB lista.

9. Ajuste del PID

Para poder ajustar el PID, es necesario comprender la función de cada una de las tres componentes de este sistema de control. He consultado [esta página](#) para comprender cómo funciona un PID. La forma de ajustar el PID está explicada en esa misma página web, pero en [otra sección](#), y la implementación en código también se explica en [otra sección](#). Cabe destacar que no consulté esta última sección.

Durante el proceso de calibración del robot, he aprendido algunos "truquitos" útiles:

- **Empezar desde 0:**

Es crucial comenzar con todos los valores desde 0 y ajustar los valores de manera ordenada e independiente.

- **Orden de calibración:**

Se recomienda iniciar ajustando los valores de **Kp** hasta lograr que el robot sea capaz de levantarse desde el suelo. Una vez logrado esto, se anota el valor que pareció óptimo y se establece la variable a 0.

Luego, se ajusta **Kd**, buscando un valor en el cual el robot no oscile exageradamente y casi se mantenga en la posición deseada.

En el tercer paso, se restablece **Kp** al valor anotado anteriormente y se deja **Kd** en el valor óptimo encontrado. En este punto, **Ki** aún se mantiene en 0, pero primero se verifica que el robot casi se mantenga en la posición deseada.

Finalmente, se ajusta **Ki**. Esta componente solo ayuda al robot a alcanzar la posición precisa; si las otras dos componentes no se han ajustado correctamente, **Ki** no cambiará radicalmente el comportamiento del robot. En resumen, **Kp** y **Ki** son las más importantes.

- **Valores pequeños:**

Los valores del PID dependen en gran medida de la construcción del robot. Es importante tener en cuenta que cuanto mayores sean los valores, más agresivo será el movimiento del robot. Se recomienda comenzar con valores menores a 1 y realizar incrementos pequeños para evitar que el robot se mueva bruscamente.

10. Programación inicial

Al combinar los códigos previamente agregados a esta documentación con un control PID, podemos lograr que el robot funcione. Sin embargo, es necesario ajustar los valores del PID para que se adapten a nuestro caso específico.

```
1 #include "Wire.h"
2 #include "I2Cdev.h"
3 #include "MPU6050.h"
4
5 MPU6050 sensor;
6
7 // Definiciones de pines de motores
8 const int pinPWMA = 16;
9 const int pinAIN2 = 17;
10 const int pinAIN1 = 18;
11 const int pinPWMB = 22;
12 const int pinBIN1 = 19;
13 const int pinBIN2 = 21;
14 const int pinSTBY = 23;
15
16
17 // Variables del sensor MPU6050
18 int16_t ax, ay, az;
19 int16_t gx, gy, gz;
20 long tiempo_prev;
21 float dt;
22 float ang_x, ang_y;
23 float ang_x_prev, ang_y_prev;
24
25 // Variables PID
26 float Kp = 0; // Ajusta estos 3 valores seg n sea necesario
27 float Ki = 0;
28 float Kd = 0;
29 float integral = 0.0;
30 float derivative = 0.0;
31 float previous_error = 0.0;
32
33 void setup() {
34   Serial.begin(115200);
35   Wire.begin(32, 33);
36   sensor.initialize();
37
38   // Configurar pines de motores
39   pinMode(pinAIN2, OUTPUT);
40   pinMode(pinAIN1, OUTPUT);
```

```

41 pinMode(pinPWMA, OUTPUT);
42 pinMode(pinBIN1, OUTPUT);
43 pinMode(pinBIN2, OUTPUT);
44 pinMode(pinPWMB, OUTPUT);
45 pinMode(pinSTBY, OUTPUT);
46 }
47
48 void loop() {
49     // Obtener datos del MPU6050
50     sensor.getAcceleration(&ax, &ay, &az);
51     sensor.getRotation(&gx, &gy, &gz);
52
53     // Calcular ngulos con el acelerometro
54     float accel_ang_x = atan(ay / sqrt(pow(ax, 2) + pow(az, 2))) * (180.0 / PI);
55     float accel_ang_y = atan(-ax / sqrt(pow(ay, 2) + pow(az, 2))) * (180.0 / PI);
56
57     // Calcular ngulo de rotacion con giroscopio y filtro complementario
58     dt = (millis() - tiempo_prev) / 1000.0;
59     tiempo_prev = millis();
60     ang_x = 0.98 * (ang_x_prev + (gx / 131.0) * dt) + 0.02 * accel_ang_x;
61     ang_y = 0.98 * (ang_y_prev + (gy / 131.0) * dt) + 0.02 * accel_ang_y;
62
63     ang_x_prev = ang_x;
64     ang_y_prev = ang_y;
65
66     // Control del pendulo invertido usando PID
67     float desired_angle = 0.0; // ngulo deseado para mantener el equilibrio
68     float error = desired_angle - ang_x; // Calcular error
69
70     integral += error * dt;
71     derivative = (error - previous_error) / dt;
72
73     float motor_speed = Kp * error + Ki * integral + Kd * derivative;
74
75     // Controlar los motores
76     enableMotors();
77     moveMotor(pinPWMA, pinAIN1, pinAIN2, motor_speed);
78     moveMotor(pinPWMB, pinBIN1, pinBIN2, motor_speed);
79
80     // Mostrar los ngulos y la velocidad de los motores
81     Serial.print("Rotacion en X: ");
82     Serial.print(ang_x);
83     Serial.print(", Error: ");
84     Serial.print(error);
85     Serial.print(", Velocidad de los motores: ");
86     Serial.println(motor_speed);
87
88     // Actualizar el error anterior
89     previous_error = error;
90
91     delay(1); // Reducir el tiempo de espera para aumentar la frecuencia de muestreo
92 }
93
94 // Funciones adicionales
95 void moveMotor(int pinPWMA, int pinIN1, int pinIN2, float speed) {
96     if (speed > 0) {
97         digitalWrite(pinIN1, HIGH);
98         digitalWrite(pinIN2, LOW);
99         analogWrite(pinPWMA, speed);
100     } else {
101         digitalWrite(pinIN1, LOW);
102         digitalWrite(pinIN2, HIGH);
103         analogWrite(pinPWMA, -speed);
104     }
105 }
106
107 void enableMotors() {
108     digitalWrite(pinSTBY, HIGH);
109 }

```

Listing 5: Código inicial completo

11. Control con mando

Este apartado aún está por desarrollarse.

Anexo B. Manual de robot Kobuki/Turtlebot2

Manual de usuario del robot Kobuki/Turtlebot2 para la extracción de parámetros de los motores del robot, para la realización del ejercicio de modelado en el espacio de estados.

Kobuki User Guide



i c l e b o
Kobuki

Version 1.1.0
2017/03/15

Contents

[Contents](#)

[About](#)

[Basic Usage](#)

[Charging](#)

[First Run](#)

[Software](#)

[Problems](#)

[Advanced Usage](#)

[Replacing Batteries](#)

[Updating Firmware](#)

[Linux](#)

[Windows](#)

[Serial Port Connectivity](#)

[Adding a Custom Sensor Array](#)

[Specifications](#)

[Safety Guideline](#)

[Functional](#)

[Hardware](#)

[Software](#)

[Anatomy](#)

[Top View](#)

[Bottom View](#)

[Control Panel](#)

[Connectors](#)

[Models & Drawings](#)

[Motors](#)

[Gyro](#)

[Power Adapter](#)

[Batteries](#)

[Serial Port](#)

[Firmware](#)

[Communication](#)

[Protocol Specification](#)

[Version Checking](#)

[Updating](#)

[Special Firmware Modes](#)

[Activating](#)

[Random Walker Mode](#)

[Arduino / Embedded Board Support Mode](#)

[ChangeLog](#)

[License](#)

About

Introducing Korea's first robotic turtle.

kobuki [거북이] n. turtle

Kobuki is robotically engineered to be long-lived, tough and fast. With high performance batteries, Kobuki will tirelessly work alongside you through those long coffee-powered nights. He'll also happily burden himself with your modded array of sensors, actuators, laptops, embedded boards, portside cannons and do it all at a speed that makes his real world cousins seem like ... well, turtles.

Use him for serving 차|맥 (chi-mek), chasing your neighbour's kids or simply, to make your own robot ideas become reality.

Kobuki is still young, don't expect him to remain as he is . Kobuki's development has already been significantly influenced by the community and as he marches towards old age, we will continue to work with the community and you to ensure he becomes better with time.

Sincerely,
Kobuki Team.

Basic Usage

Make sure be aware of the [safety guidelines](#).

Charging

Connect the power adapter to Kobuki or dock Kobuki in the docking station. If Kobuki is turned on, you will hear a short sound when charging starts and the LED will light up appropriately.

- Green : fully charged
- Blinking Green : charging
- Orange : low battery

Tip: the battery still charges if Kobuki is off, but you will not see the LED, nor hear sounds

First Run

You want to see Kobuki in action without further ado? Kobuki has a special random walker mode embedded into the firmware which you can activate on start-up:

1. Turn on Kobuki.
2. Within the first 3 seconds press the B0 button and hold for 2 seconds.
3. LED2 will start blinking and Kobuki wander around.

Tip: this was introduced to the firmware in v1.1.0. In case your kobuki is not running this or a later version, please refer to the section about [updating the firmware](#).

Software

In order to gain access to all of Kobuki' s features, you will need an external computing unit (pc/laptop/tablet/embedded board) with software which communicates with Kobuki. Officially supported systems include:

Build Your Own Kobuki Driver Use the Serial Protocol	Linux C++ Application Use the Kobuki C++ Driver	Kobuki for ROS Use the Kobuki ROS World
--	---	---

Alternatively, you can get started with the [Turtlebot 2 Platform](#) which runs on a Kobuki base.

Info: other platforms have had experimental support at one stage or another (windows/arduino/android).

Problems

Reach out on the following channels:

- Kobuki User's Mailing List: kobuki-users@yujinrobot.com
- ROS Community Answers: <http://answers.ros.org/questions/tags%3Akobuki>
- Yujin Sales Support: kobuki@yujinrobot.com

Note: Kobuki comes with a 1 year warranty. In case you bought Kobuki from another one of Yujin Robot's distributors, please contact them, otherwise email Yujin Sales Support.

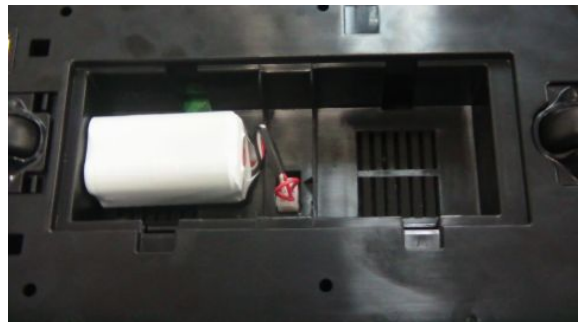
Advanced Usage

Replacing Batteries

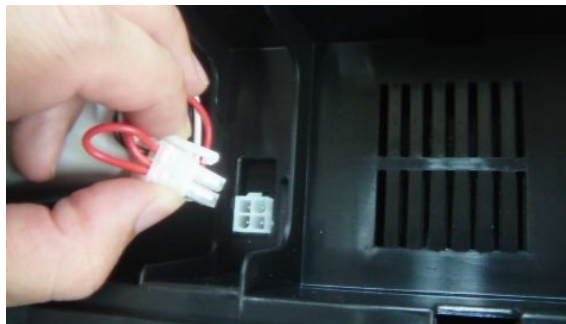
Warning: the electronics does not support the use of multiple battery packs at the same time!



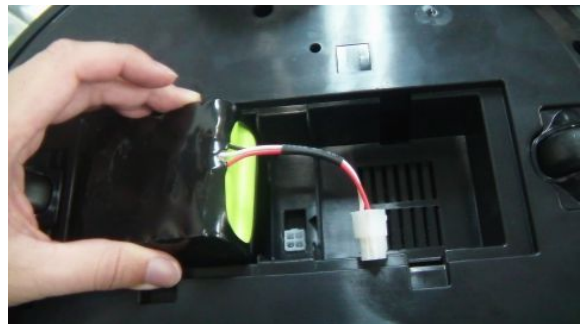
Flip the kobuki



Open the case



Unplug



Swap batteries



Replug



Replace the case

Updating Firmware

If you have a ROS installation, you can find the current version by running:

```
$ rosrun kobuki_driver version_info
# Version Info:
* Hardware Version: 1.0.4
* Firmware Version: 1.2.0
* Software Version: 0.7.7
* Unique Device ID: 97713968-842422349-1361404194
```

Alternatively you can compile [version_info.cpp](#) (no ROS) to create your own version checker.

Linux

The Flashing Utility

```
# Download & build the flashing utility
$ wget http://files.yujinrobot.com/kobuki/firmware/stm32flash-0.4.tar.gz
$ tar -xvzf stm32flash.tar.gz
$ cd stm32flash
$ make
```

Download the Firmware

```
# choose the firmware version from http://files.yujinrobot.com/kobuki/firmware
# e.g. latest
$ wget http://files.yujinrobot.com/kobuki/firmware/kobuki_firmware-latest.hex
```

Identify the ttyUSBx Port

If you have a udev rule installed, it will show up as `/dev/kobuki`, but if not, you can typically find it under one of the `ttyUSB` ports, e.g. `/dev/ttyUSB0`. If you are not sure, type `dmesg` into a terminal, unplug and replug the robot and type `dmesg` again. You should now be able to see which port is assigned to the robot (more precisely the FTDI USB to serial converter).

Switch to Download Mode

1. Connect the robot to your PC using the USB cable
2. Turn off the robot (switch on the side)
3. Switch from normal runtime mode to firmware download mode

This simply changes the type of data that is sent back and forth along the usb connection. You can do this by moving the switch illustrated below into the 'download' (up) position. Note that this switch is embedded into the robot cover so it isn't easily thrown by accident – you may need thin plyers or some similar tool. You can find the mode switch mechanism on the right side of the control panel - see the image below.



Flashing

Note: The following instructions assume flashing of `kobuki_firmware-latest.hex` and port `/dev/ttyUSB0`. Modify these if necessary.

Warning: you need to execute the flashing command IMMEDIATELY after turning the robot on!

1. Turn off the robot
2. Check that the switch is in download mode
3. Turn on the robot

```
$ ./stm32flash -b 115200 -w kobuki_firmware_firmware-latest.hex /dev/ttyUSB0
stm32flash 0.4

http://stm32flash.googlecode.com/

Using Parser : Intel HEX
Interface serial_posix: 115200 8E1
Version      : 0x22
Option 1     : 0x00
Option 2     : 0x00
Device ID    : 0x0414 (High-density)
- RAM        : 64KiB (512b reserved by bootloader)
- Flash      : 512KiB (sector size: 2x2048)
- Option RAM : 16b
- System RAM : 2KiB
Write to memory
Erasing memory
Wrote address 0x0800a3f0 (100.00%) Done.
```

Rebooting

- Turn off the robot power
- Flick the firmware switch back to 'Operation' mode.
- Turn on the robot power
- I'm happy, you should be too!

Windows

The Flashing Utility

- Download and install the [windows flashing utility](#)

Download the Firmware

Find and download the version of the firmware you wish to flash from the [Kobuki FW File Server](#).

Identify the COM Port

Usually if this will show up on COM1, but check to make sure.

Switch to Download Mode

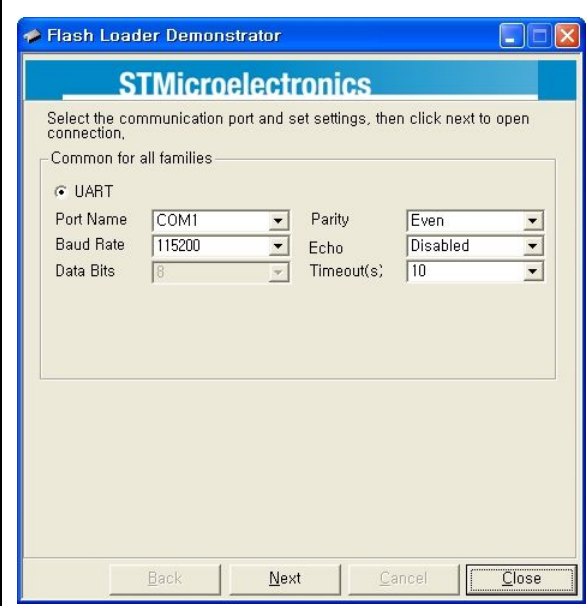
1. Connect the robot to your PC using the USB cable
2. Turn off the robot (switch on the side)
3. Switch from normal runtime mode to firmware download mode

This simply changes the type of data that is sent back and forth along the usb connection. You can do this by moving the switch illustrated below into the 'download' (up) position. Note that this switch is embedded into the robot cover so it isn't easily thrown by accident – you may need thin plyers or some similar tool. You can find the mode switch mechanism on the right side of the control panel - see the image below.



Flashing

1. Turn off the robot
2. Check that the switch is in download mode
3. Turn on the robot



Flash Loader Demonstrator

STMicroelectronics

Select the communication port and set settings, then click next to open connection.

Common for all families

☒ UART

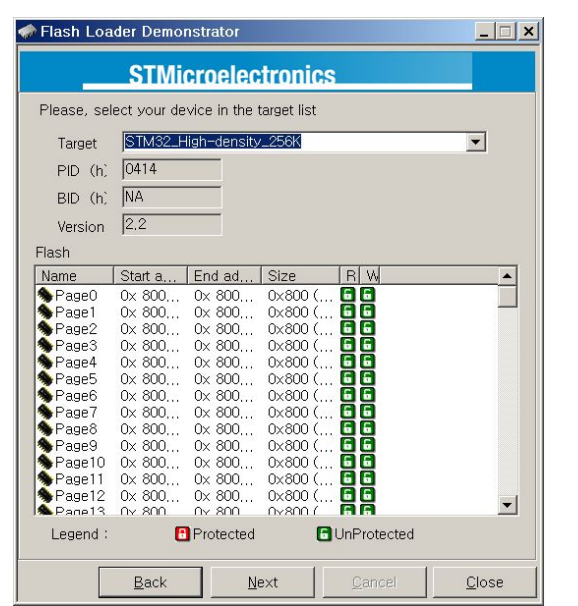
Port Name: COM1 Parity: Even

Baud Rate: 115200 Echo: Disabled

Data Bits: 8 Timeout(s): 10

Back Next Cancel Close

Configure Properties



Flash Loader Demonstrator

STMicroelectronics

Please, select your device in the target list

Target: STM32_High-density_256k

PID (h): 0414

BID (h): NA

Version: 2.2

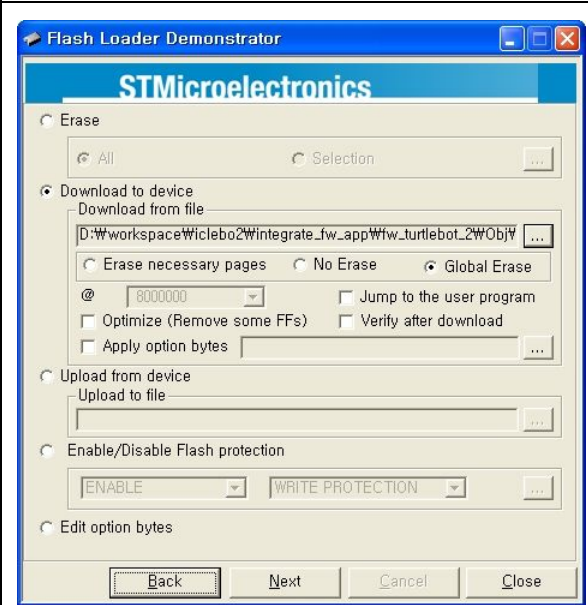
Flash

Name	Start address	End address	Size	R	W
Page0	0x 800...	0x 800...	0x800 (...)		
Page1	0x 800...	0x 800...	0x800 (...)		
Page2	0x 800...	0x 800...	0x800 (...)		
Page3	0x 800...	0x 800...	0x800 (...)		
Page4	0x 800...	0x 800...	0x800 (...)		
Page5	0x 800...	0x 800...	0x800 (...)		
Page6	0x 800...	0x 800...	0x800 (...)		
Page7	0x 800...	0x 800...	0x800 (...)		
Page8	0x 800...	0x 800...	0x800 (...)		
Page9	0x 800...	0x 800...	0x800 (...)		
Page10	0x 800...	0x 800...	0x800 (...)		
Page11	0x 800...	0x 800...	0x800 (...)		
Page12	0x 800...	0x 800...	0x800 (...)		
Page13	0x 800...	0x 800...	0x800 (...)		

Legend: Protected UnProtected

Back Next Cancel Close

Check that the target is identified



Flash Loader Demonstrator

STMicroelectronics

☐ Erase

☒ Download to device

Download from file

D:\workspace\wiclebo2\integrate_fw_app\fw_turtlebot_2\Obj\...

☐ Erase necessary pages ☐ No Erase ☒ Global Erase

@ 8000000 ☐ Jump to the user program

☐ Optimize (Remove some FFs) ☐ Verify after download

☐ Apply option bytes

☐ Upload from device

Upload to file

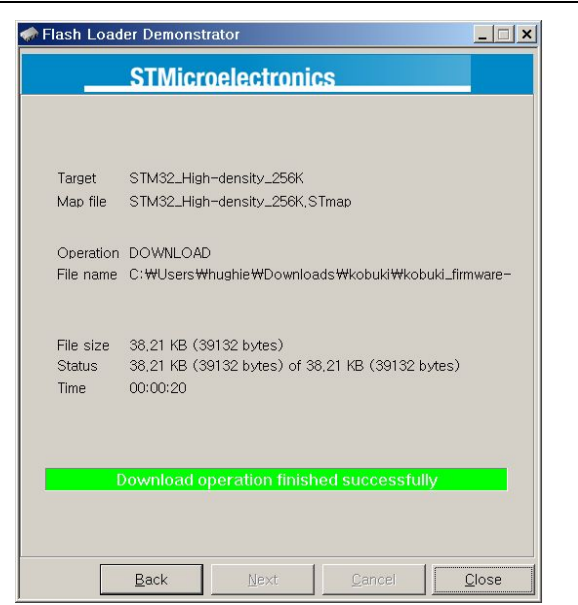
☐ Enable/Disable Flash protection

ENABLE WRITE PROTECTION

☐ Edit option bytes

Back Next Cancel Close

Enter the Download from file (your .hex)



Flash Loader Demonstrator

STMicroelectronics

Target: STM32_High-density_256K

Map file: STM32_High-density_256K.STmap

Operation: DOWNLOAD

File name: C:\Users\Wuhugie\Downloads\Wkobuki\Wkobuki_firmware-

File size: 38,21 KB (39132 bytes)

Status: 38,21 KB (39132 bytes) of 38,21 KB (39132 bytes)

Time: 00:00:20

Download operation finished successfully

Back Next Cancel Close

Success!

Rebooting

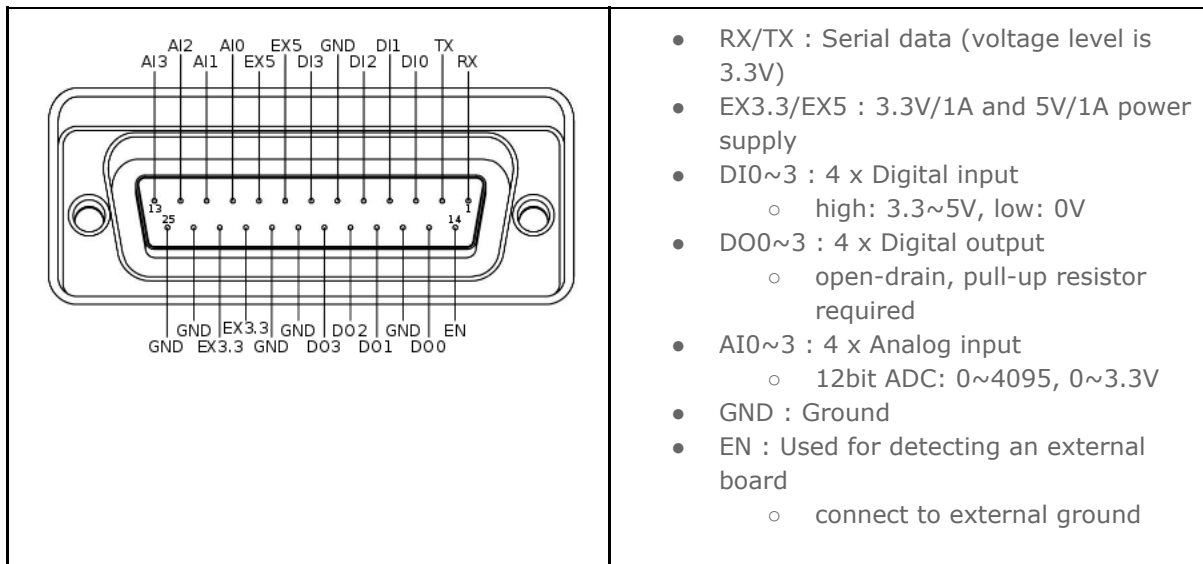
1. Turn off the robot power
2. Flick the firmware switch back to 'Operation' mode.
3. Turn on the robot power
4. I'm happy, you should be too!

Serial Port Connectivity

Many embedded boards do not have usb interfaces, or the embedded board doesn't have the necessary ftdi32 serial->usb support. In these cases it is simpler to utilise the serial port on Kobuki's expansion port to driving kobuki by serial interface instead of USB.

Pinouts

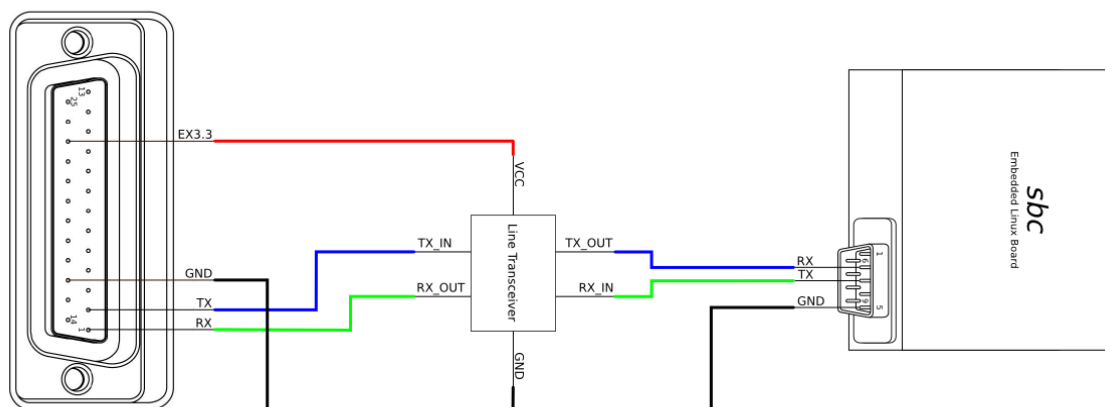
Pictured below are the pinouts of Kobuki's expansion port, include the serial pins. The minimum number of required pins for serial communication is three; TX, RX, and GND. Additionally EX3.3 or EX5 can be used for powering external devices, such as line transceiver.



Connections

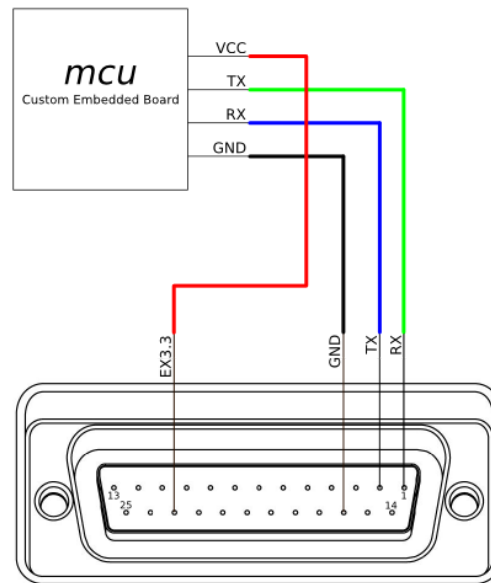
a) RS-232 Interface

The voltage level of serial port is 3.3V. To connect kobuki with the standard [RS-232](#) serial port, you should use line transceiver in the middle to convert voltage level. [MAX232](#) chip is typical solution for it. Below diagram show simplified typical example of connections between Kobuki and [SBC](#) (your embedded board, laptop or pc). EX3.3 pin is used to powering line transceiver. It can be EV5 pin or external power sources from outside.



b) Logic Level Interface

To connect kobuki with serial port of [MCU](#) directly, below is the simplest connection diagram can be used. Serial pins of kobuki are 5V-tolerant. It accepts 3.3V and 5V voltage levels typically used by integrated circuit. Connection with power pin(EX3.3 or EX5) is optional, if you powering MCU with independent sources, such as battery, or wall adaptor.



Building An Application

If you're using the ROS implementation, or building atop the kobuki c++ driver, simply re-configure the port string used for the application and everything should be ready to go.

If building an **experimental driver** from scratch (e.g. android driver), then you will need to implement the [Kobuki Protocol](#) specification.

Adding a Custom Sensor Array

Kobuki usually gets equipped with a 3d sensor. These typically have limitations as obstacle avoidance sensor:

- Narrow fov (58° x 43° horizontal x vertical)
- Death zone on the first 45 cm
- Cannot detect glass walls
- Cannot detect polished metallic surfaces

In one experiment an 11 IR sensors half ring, pointing 12 degrees downward was added to compensate.

- **Sensor model:** Sharp GP2Y0A21YK
- **Power supply:** Kobuki's 5V, 1A
- **Sensor reading:** Arduino MEGA 2560
- **PC interface:** Arduino custom firmware – Bosch adc_driver
- **Mounting:** 3D printed frame.

The analog output of sensors is read by the Arduino board, while for power and ground they are connected to Kobuki's 5V 1A power source. Connecting several sensors to the same power supply makes readings very noisy when there aren't obstacles. The solution was to put decoupling capacitors on each sensor. For interfacing Arduino, we use Bosch adc_driver. A funny problem comes sometimes when looking at corners in a particular angle (~30 degrees) : sensors get a spurious reading, no idea why. We just went on with this, as is not a common problem.

Different mounting frames are available for downloading and printing in our [file server](#):

- Horizontally mounted MaxBotix's LV-Maxsonars
- Horizontally mounted Sharp IR sensors
- 12 degrees downward pointing Sharp IR sensors

And the result:



Specifications

Safety Guideline

- Do not twist or subject the power cable to extreme pressure or weight weight.
- Keep the pin and interface of the power plug clean from dust or water.
- Do not pull the power cord or touch the power plug with wet hands.
- Do not use with a damaged power plug, power cord or loose outlet.
- Use Kobuki indoors only.
- Do not pour or spray water onto Kobuki.
- Do not use Kobuki to pick up anything that is burning or smoking.
- Always remove the battery before long-term storage or transportation.

Functional

- Maximum translational velocity: 70 cm/s

- Maximum rotational velocity: 180 deg/s (>110 deg/s gyro performance will degrade)
- Payload: 5 kg (hard floor), 4 kg (carpet)
- Cliff: will not drive off a cliff with a depth greater than 5cm
- Threshold Climbing: climbs thresholds of 12 mm or lower
- Rug Climbing: climbs rugs of 12 mm or lower
- Expected Operating Time: 3/7 hours (small/large battery)
- Expected Charging Time: 1.5/2.6 hours (small/large battery)
- Docking: within a 2mx5m area in front of the docking station

Hardware

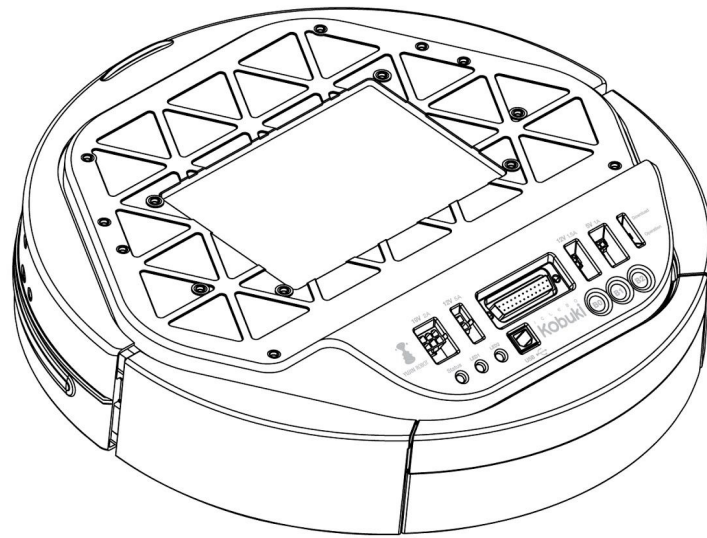
- PC Connection: USB or via RX/TX pins on the parallel port
- Motor Overload Detection: disables power on detecting high current (>3A)
- Odometry: 52 ticks/enc rev, 2578.33 ticks/wheel rev, 11.7 ticks/mm
- Gyro: factory calibrated, 1 axis (110 deg/s)
- Bumpers: left, center, right
- Cliff sensors: left, center, right
- Wheel drop sensor: left, right
- Power connectors: 5V/1A, 12V/1.5A, 12V/5A
- Expansion pins: 3.3V/1A, 5V/1A, 4 x analog in, 4 x digital in, 4 x digital out
- Audio : several programmable beep sequences
- Programmable LED: 2 x two-coloured LED
- State LED: 1 x two coloured LED [Green - high, Orange - low, Green & Blinking - charging]
- Buttons: 3 x touch buttons
- Battery: Lithium-Ion, 14.8V, 2200 mAh (4S1P - small), 4400 mAh (4S2P - large)
- Firmware upgradeable: via usb
- Sensor Data Rate: 50Hz
- Recharging Adapter: Input: 100-240V AC, 50/60Hz, 1.5A max; Output: 19V DC, 3.16A
- Netbook recharging connector (only enabled when robot is recharging): 19V/2.1A DC
- Docking IR Receiver: left, centre, right

Software

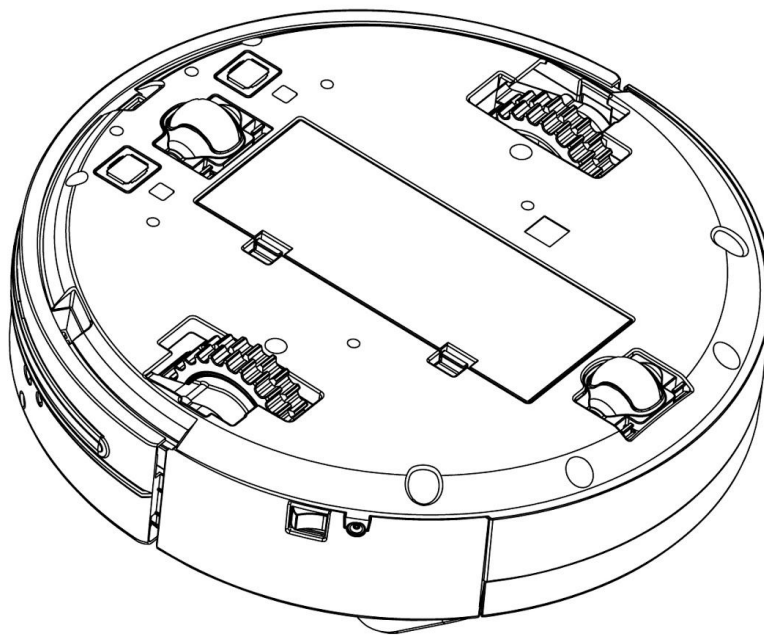
- C++ drivers for Linux
- ROS driver
- Gazebo simulation

Anatomy

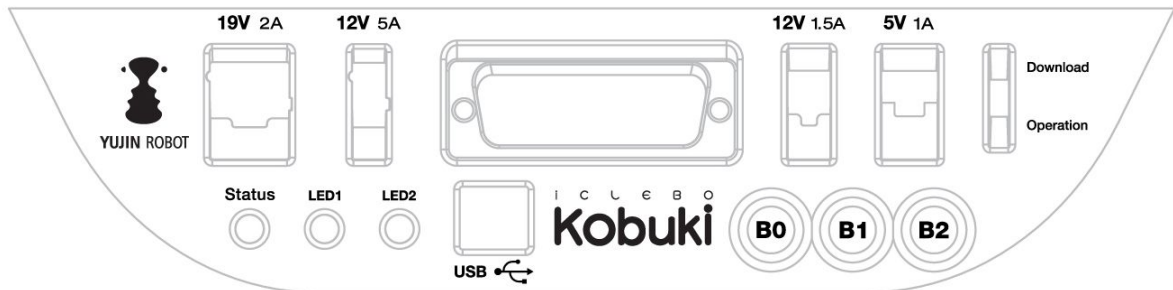
Top View



Bottom View



Control Panel



- 19V/2A: Laptop power supply
- 12V/5A: Arm power supply
- 12v/1.5A: Microsoft Kinect power supply
- 5V/1A: General power supply
- Status LED: Indicates Kobuki's status
 - Green: Kobuki is turned on and battery at high voltage level
 - Orange: On - Low battery voltage level (please charge soon)
 - Green blinking: On - Battery charging
 - Off: Kobuki is turned off.
- LED1/2: Programmable LEDs
- USB: Data connection
- BO/1/2: Buttons
- Firmware switch: Enable/disables the firmware update mode

Connectors

SOME NOTES ABOUT THE MOLEX PAGES BELOW

1. We do not actually use Molex connectors but we are supplied by a Korean vendor who produces connectors according to the Molex standard. These links will be more useful to internationals in helping them find a mating part that works for them.
2. The images on each page are representative of the series of connectors. Each series usually has a variety of connectors with a different number of pins. As a result, the pictures on some of the pages below may seem as though they have the incorrect number of pins, but do not worry about this – they are the correct links. Note that you can jump to different connectors in the series via the second part of their identification number (e.g. 43045-0224 for the 2-pin, 43045-0424 for the 4-pin).
3. If some linked connectors are listed as obsolete on the molex website, don't worry. The connector you are exactly requiring are those you can find under the 'Mates with Parts' link on each page. If these however should become obsolete as well, please let us know via email.

Power

- 5V@1A Molex PN : 43650-0218 – for custom boards
- 12V@1.5A : Molex PN : 43045-0224 – specially supporting the kinect
- 12V@5A : Molex PN : 3929-9023 – for high powered accessories (e.g. robotic arm)
- 19V@2A : Molex PN : 3928-9068 – for recharging netbooks



Battery

- [4S1P/4S2P Battery Pack Connector](#)

I/O Port

- DB25 pin D-SUB Female connector that provides the following functionality ([pdf](#))

Cables

Note, if you click on the preceding links for the power connectors, under the heading 'Mates with Part(s)' you can find the compatible connector to use with each power source. The most important one being of course:

- 12V@1.5A : [Molex PN : 43025-0200](#) – specially supporting the kinect

Models & Drawings

The models and drawings include both the base and parts for the Turtlebot 2.

- [2D mechanical drawings](#) – DWG, PDF
- [3D models](#) – IGS, STEP

The inserts in the kobuki plate are M4 threads (metric, 4mm). If you wish to build standoffs compatible for these inserts, please reference the pole pdf's in the 2D mechanical drawings which are what we use for turtlebots.

Motors

Specifications

- Brushed DC Motor
- Motor Manufacturer: Standard Motor
- Part Name: RP385-ST-2060
- Rated Voltage: 12 V
- Rated Load: 5 mN·m
- No Load Current: 210 mA
- No Load Speed: 9960 rpm $\pm$ 15%
- Rated Load Current: 750 mA
- Rated Load Speed: 8800 rpm $\pm$ 15%
- Armature Resistance: 1.5506 Ω at 25°C
- Armature Inductance: 1.51 mH
- Torque Constant(Kt): 10.913 mN·m/A

- Velocity Constant(Kv): 830 rpm/V
- Stall Current: 6.1 A
- Stall Torque: 33 mN·m

Control Method

- Driven by voltage source(H-bridge)
- Controlled by Pulse-width modulation(PWM)

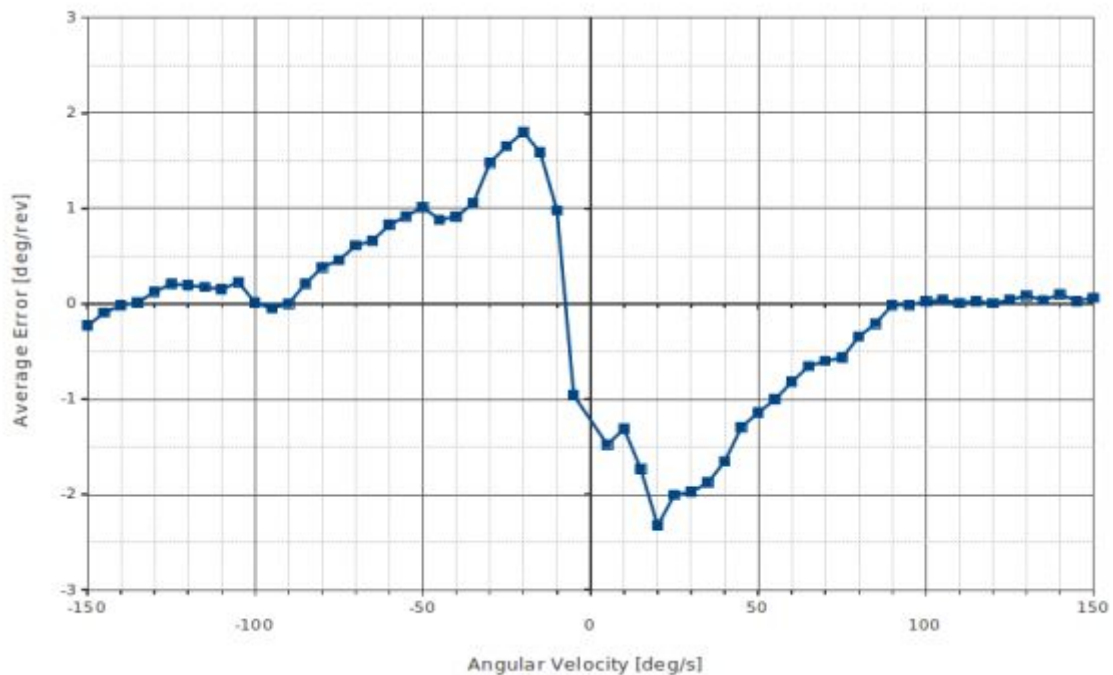
Gyro

Specification

- 3-Axis Digital Gyroscope
- Manufacturer : STMicroelectronics
- Part Name : [L3G4200D](#)
- Measurement Range: ± 250 deg/s
- Yaw axis is factory calibrated within the range of ± 20 deg/s to ± 100 deg/s

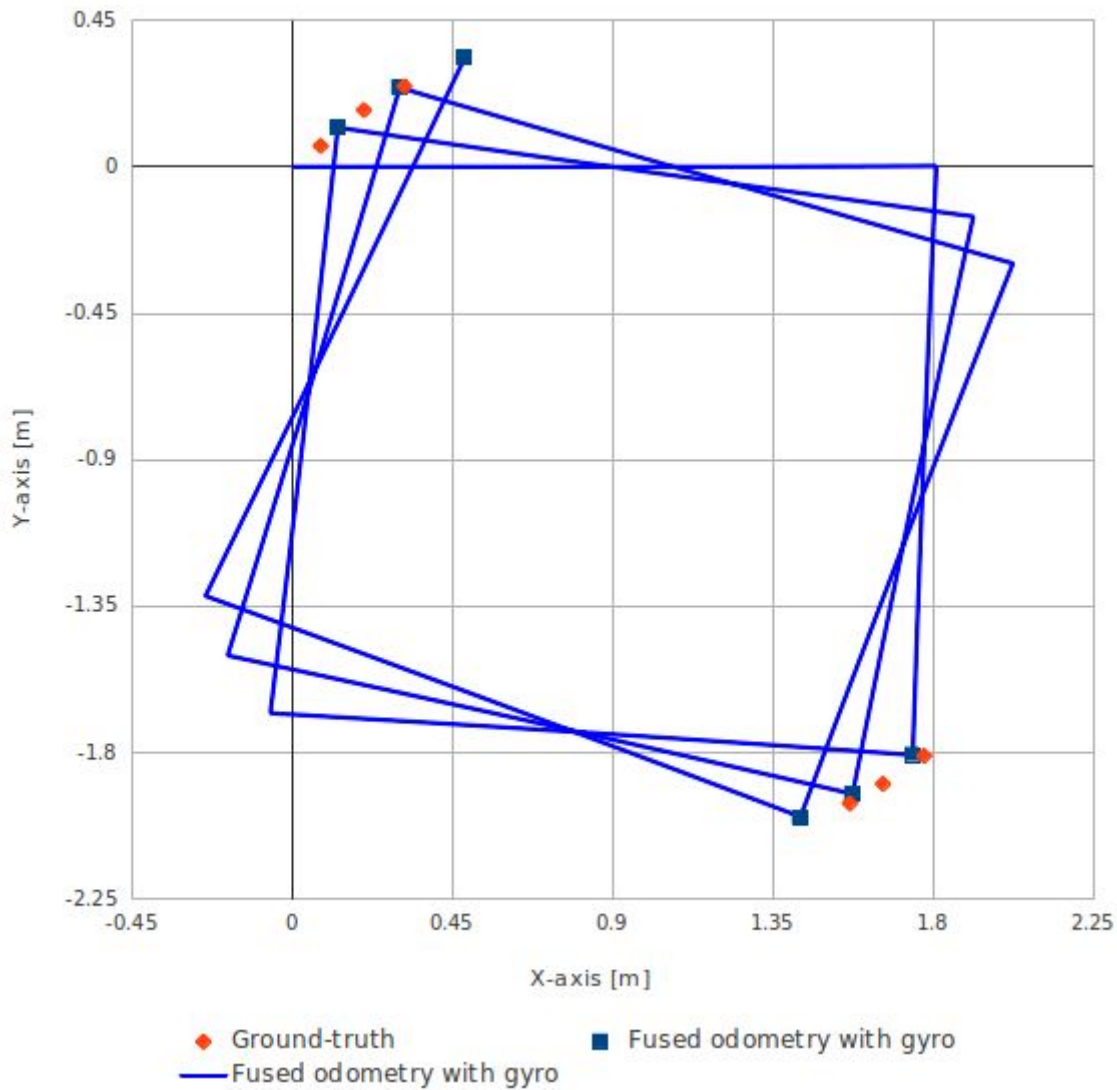
Performance

In-Place Rotation Test



This graph shows the average heading error per revolution of gyro, when robot rotates with a given velocity.

Square Path Test



This graph shows the position error of fused odometry with gyro, when robot moves along a square path. Robot moved with 0.1 m/s on the line segment and rotated with 30 deg/s on the corner.

Number of turns of square path	Angular Error [deg]
0.5	0.47
1.5	1.99
2.5	3.18

This table shows the calculated angular error, when robot arrived at the diagonally opposite corner from the starting point (0.0, 0.0).

Power Adapter

Input	Output	Other
- Voltage: 100-240V - Frequency: 50/60Hz - Ampere: 1.5A Max	- Voltage: 19V - Ampere: 3.16A	Photo Specification 1 Specification 2

Batteries

Kobuki by the default ships with a small Lithium-Ion battery pack (4S1P, 2200mAh, 14.8V).

Tip: for extra long operation, a big battery pack (4S2P, 4400mAh, 14,8V) can be ordered as well.

Warning: the electronics does not support the use of multiple battery packs at the same time (even if there is room in the battery compartment)

Specifications

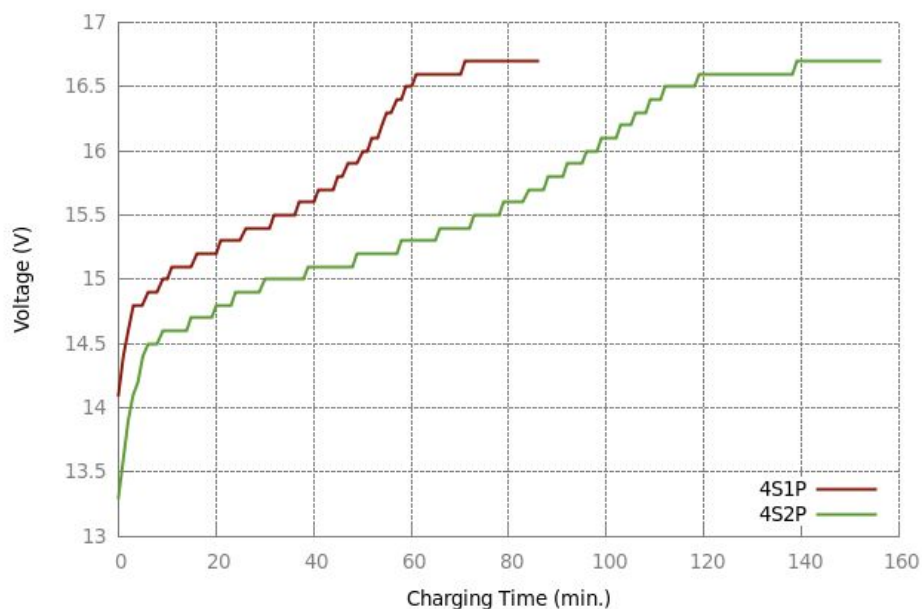
- [Data Sheet \(pdf\)](#)

Pinouts

- **Red:** battery (+), 9.6 V ~ 16.8 V
- **White:** NTC thermistor to ground, 10 k Ω $\pm$ 1%
- **Black:** battery(-), Ground

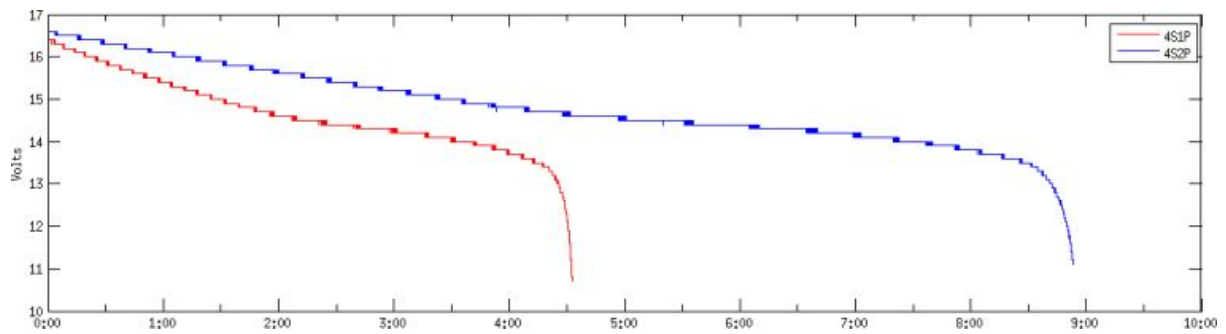
Charging Profile

This plot shows the voltages as measured by the robot's hardware. Both the standard 4S1P and the extra 4S2P batteries are compared. During the test, the robot was charging via adaptor.

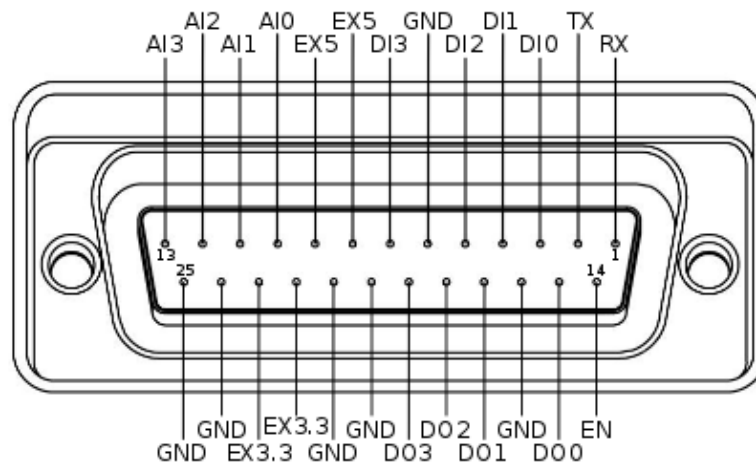


Discharging Profile

This plot shows the voltage as measured by the robot's hardware. Both the standard 4S1P and the extra 4S2P batteries are compared. During the test, the robot was continuously spinning, with the Kinect camera running.



Serial Port



- RX / TX: Serial data connection (RS232; used voltage level is 3.3V!)
- EX3.3 / EX5: 3.3V/1A and 5V/1A power supply
- DI0 - 3: 4 x Digital input (high: 3.3 - 5V, low: 0V)
- DO0 - 3: 4 x Digital output (open-drain, pull-up resistor required)
- AI0-3: 4 x Analog input (12bit ADC: 0 - 4095, 0 - 3.3V)
- GND: Ground
- EN: Used for detecting an external board (connect to external ground)

Firmware

Communication

- Baud rate: 115200 BPS, Data bit: 8 bit, Stop bit: 1 bit, No Parity
- Electronic protocol: USB / RS232 (@ DB25 connector)

Protocol Specification

The driver communicates with the robot by using predefined protocol. In general, the driver sends the commands to the robot and the robot sends some feedback data or sensor readings. This command and feedback data are converted into bytestreams for communication via serial interface. The protocol specifies the rules and forms of bytestream.

For more detailed information, visit:

- <http://yujinrobot.github.com/kobuki/enAppendixProtocolSpecification.html>

Version Checking

Every time the robot comes alive (because it is connected or powered on), the driver checks the compatibility between software (i.e. driver) and firmware. Firmware versions are of the form M.m.p, where:

- M(ajor) represents a deep rebuild of the code that almost surely breaks protocol compatibility. In consequence, if major version doesn't match, the driver will show an error, suggest the required update and shutdown.
- m(inor) represents a new feature that could not work if the driver is outdated, but the protocol itself is spared. In consequence, if minor version doesn't match, the driver will show a warning suggesting the required update and continue working.
- p(atch) represents a fix on the code and is not checked at all.

You can check which firmware version your robot is running on the log (check the first lines on stdout at driver startup), by running the version_info program included on the kobuki driver, or if you are using ROS, just echoing /mobile_base/version_info topic. Generally speaking, it's recommended to upgrade the firmware to the latest stable version.

Special Firmware Modes

Activating

Kobuki has some special firmware modes, which can be activated on startup. Currently implemented are:

- I. Random walker mode
- II. Arduino/Embedded board support mode

To activate one of them, follow these instructions:

1. Turn on Kobuki.

2. Within in the first 3 seconds press and hold button BO(I) / B1 (II) for 2 seconds.
3. If* you see LED2(I) / LED1(II) switching between red and green, your chosen mode is activated.

Note(*): These modes have been introduced to the firmware with version 1.1.0. In case your Kobuki is not running this or a later version, please refer to the section about [updating the firmware](#).

Random Walker Mode

In the random walker mode Kobuki is driving around until it hits an object with the bumper or a cliff is detected. In both cases, Kobuki will stop, turn by a random amount of degrees and continue driving .

Note: In this mode Kobuki's wheel drop sensors are not activated. So, be careful when lifting up Kobuki!

Arduino / Embedded Board Support Mode

In this mode the serial port (DB25 connector) gives access to basic controls of Kobuki. You can hook up the digital/analog inputs/outputs of your Arduino or other embedded boards and start writing simple control programs.

Below is the special pin setting listed. Please refer to the [serial port description](#) for the name to pin mapping.

- DI0: Not used
- DI1: Not used
- DI2: Not used
- DI3: Not used
- DO0: Bumper left (pressed/released)
- DO1: Bumper centre (pressed/released)
- DO2: Bumper right (pressed/released)
- DO3: Wheel drop sensors (at least one wheel is dropped / none is dropped)
- AI0: Wheel speed right (0V - full speed backward, 3.3V - full speed forward)
- AI1: Wheel speed left (0V - full speed backward, 3.3V - full speed forward)
- AI2: Not used
- AI3: Not used

All other pins (GND, RX, TX etc.) remain unchanged.

Note: To enable the motors you need to press button B0.

Resources

- [Kobuki Images & Renderings](#)
- [Turtlebot Images & Renderings](#)
- [Marketing Materials](#)

ChangeLog

- 1.0.0 : official release
- 1.0.1 : upgrading firmware link fix
- 1.0.2 : documentation link fix
- 1.0.4 : updating the spec. of serial communication
- 1.0.5 : fixing battey & connector link
- 1.0.6 : updating link after Kobuki web site revision
- 1.1.0 : reorganised and collapsed website howtos into the guide

License

The Kobuki user guide is licensed under a [Creative Commons Attribution-ShareAlike 3.0 Unported License](#).

